ULG Triad Spec v0.5

PeerCompute + Eshkol + Moonlab Integration

2026-06-05

Contents

1 Part 0 — README / Executive Overview

Version: **0.5**
Date: **2026-06-05**

1.1 Abstract

This v0.5 specification defines a combined browser-native scientific-computing stack built from **PeerCompute**, **Eshkol**, **Moonlab**, and a Universal Law Graph (**ULG**) simulation runtime. The goal is to let distributed browser peers run multiscale physics workloads where low-level quantum and mathematical tasks generate validated closures that are consumed by scalable WebGPU simulation kernels.

For readers unfamiliar with the three libraries: **PeerCompute** is treated as the browser-based P2P orchestration layer. It owns session topology, distributed task scheduling, shared hot/warm/cold state, compute-worker supervision, validation, artifact caching, and visualization. **Eshkol** is treated as a Scheme-like mathematical/law language for automatic differentiation, free-energy and response-function authoring, closure derivation, and WASM/WebGPU-compatible numerical compilation. **Moonlab** is treated as the quantum and many-body solver backend: it produces spectra, response samples, state-vector/tensor-network results, Hamiltonian evaluations, and CPU/WebGPU parity reports used by Eshkol and ULG. In this design, **PeerCompute is the authority**, while **Eshkol and Moonlab run as supervised WASM/WebGPU compute services** that may spawn child workers only through PeerCompute leases.

The plan assumes all three libraries will be extended. PeerCompute gains service registration, worker-tree supervision, GPU/resource leasing, closure/artifact registries, and NetViz extensions. Eshkol gains ULG law macros, closure manifests, derivative metadata, Moonlab task invocation, WGSL/table-generation targets, and WASM reference exports. Moonlab gains ULG quantum-task APIs, response-artifact schemas, WebGPU kernel expansion, and deterministic CPU/GPU parity reporting. A shared **ULG GPU ABI** connects all three so tensors, complex numbers, closure tables, worker messages, tolerance reports, and provenance blocks have one common representation.

1.2 Files

1. `01_SYSTEM_SPEC.md` — normative runtime, service, artifact, and resource-control specification.
2. `02_ARCHITECTURE_PLAN.md` — component architecture, execution flows, and data lifecycle.
3. `03_IMPLEMENTATION_ROADMAP.md` — phased implementation plan with acceptance tests.
4. `04_LIBRARY_EXTENSION_PLAN.md` — concrete extension plan for PeerCompute, Eshkol, and Moonlab.
5. `COMBINED_SPEC.md` — all documents merged into one Markdown file.
6. `diagrams/` — standalone Mermaid source files.
7. `schemas/` — initial JSON schema sketches.

1.3 v0.5 change summary

- Adds a formal abstract for readers unfamiliar with PeerCompute, Eshkol, Moonlab, and ULG.
- Promotes PeerCompute from generic compute host to the explicit orchestration authority.
- Defines Moonlab and Eshkol as supervised WASM/WebGPU compute services that may spawn child workers only through PeerCompute leases.
- Adds a shared ULG GPU ABI and ULG IR boundary so all three libraries exchange tensors, complex arrays, closure tables, kernel modules, tolerance reports, and provenance blocks consistently.
- Adds schema sketches for service manifests, task capsules, closure artifacts, GPU leases, and validation reports.
- Expands the plan for extending all three libraries.

1.4 System context

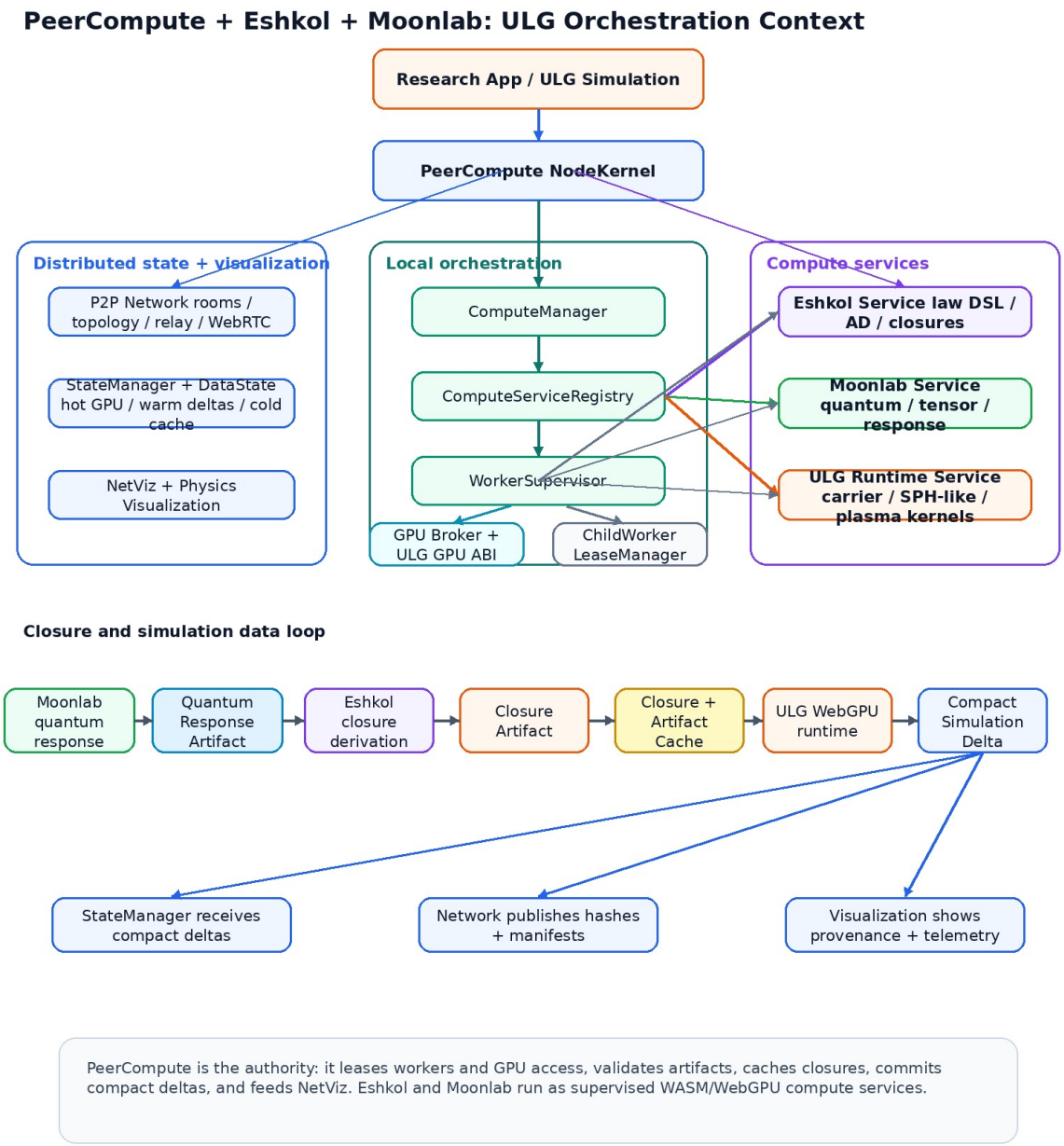


Figure 1: System context and orchestration loop

1.5 One-sentence architecture

PeerCompute supervises distributed worker trees and visualization; Eshkol derives and compiles physics closures; Moonlab supplies quantum/many-body response artifacts; ULG/WebGPU workers consume validated closures to run scalable carrier simulations.

1.6 Source notes

This specification is based on the public project documentation checked on 2026-06-05:

- PeerCompute repository and README

- Eshkol repository and README
- Moonlab repository and README
- Moonlab architecture notes
- Moonlab WebGPU plan
- Web Workers documentation
- Worker WebGPU entry point
- SharedArrayBuffer isolation requirements
- WGSL specification

2 Part 1 — System Spec

Version: **0.5**
Date: **2026-06-05**
Status: **draft specification**

2.1 Abstract

This v0.5 specification defines a combined browser-native scientific-computing stack built from **PeerCompute**, **Eshkol**, **Moonlab**, and a Universal Law Graph (**ULG**) simulation runtime. The goal is to let distributed browser peers run multiscale physics workloads where low-level quantum and mathematical tasks generate validated closures that are consumed by scalable WebGPU simulation kernels.

For readers unfamiliar with the three libraries: **PeerCompute** is treated as the browser-based P2P orchestration layer. It owns session topology, distributed task scheduling, shared hot/warm/cold state, compute-worker supervision, validation, artifact caching, and visualization. **Eshkol** is treated as a Scheme-like mathematical/law language for automatic differentiation, free-energy and response-function authoring, closure derivation, and WASM/WebGPU-compatible numerical compilation. **Moonlab** is treated as the quantum and many-body solver backend: it produces spectra, response samples, state-vector/tensor-network results, Hamiltonian evaluations, and CPU/WebGPU parity reports used by Eshkol and ULG. In this design, **PeerCompute is the authority**, while **Eshkol and Moonlab run as supervised WASM/WebGPU compute services** that may spawn child workers only through PeerCompute leases.

The plan assumes all three libraries will be extended. PeerCompute gains service registration, worker-tree supervision, GPU/resource leasing, closure/artifact registries, and NetViz extensions. Eshkol gains ULG law macros, closure manifests, derivative metadata, Moonlab task invocation, WGSL/table-generation targets, and WASM reference exports. Moonlab gains ULG quantum-task APIs, response-artifact schemas, WebGPU kernel expansion, and deterministic CPU/GPU parity reporting. A shared **ULG GPU ABI** connects all three so tensors, complex numbers, closure tables, worker messages, tolerance reports, and provenance blocks have one common representation.

2.2 1. Purpose and scope

This document specifies the runtime contracts for integrating PeerCompute, Eshkol, Moonlab, and ULG into a distributed browser-native scientific-computing system.

The design assumes all three upstream libraries will be extended. It does not assume Eshkol or Moonlab replace PeerCompute. PeerCompute remains the scheduling, network, state, validation, visualization, and local resource authority. Eshkol and Moonlab become compute services managed by PeerCompute.

2.2.1 In scope

- PeerCompute compute-service registration.
- Long-lived WASM/WebGPU service workers.
- Supervised child workers for Moonlab and Eshkol.
- GPU/resource leases.
- ULG GPU ABI and closure artifact contracts.
- Quantum response artifacts from Moonlab.
- Closure derivation artifacts from Eshkol.
- Validation, provenance, caching, visualization, and distributed publication.

2.2.2 Out of scope for v0.5

- A complete physics implementation.
- A complete DFT/QED/nuclear solver.
- Browser security bypasses or unsupervised remote code execution.

- Full-state replication of large tensors or simulation domains.
- Native-cluster orchestration outside PeerCompute's browser-first model.

2.3 2. System roles

2.3.1 2.1 PeerCompute

PeerCompute is the orchestration authority. It hosts:

- `NodeKernel` session authority;
- P2P room/topology/network scheduling;
- shared state via hot/warm/cold `DataState` layers;
- compute service registry;
- worker supervisor and child-worker lease manager;
- GPU broker;
- artifact and closure cache;
- validation manager;
- task lineage tracker;
- NetViz and physics telemetry.

PeerCompute should evolve from a flat task runner into a **supervised distributed worker-tree runtime**.

2.3.2 2.2 Eshkol

Eshkol is the law and closure compiler. It hosts:

- ULG law macros;
- free-energy, response-function, and EOS definitions;
- symbolic/forward/reverse AD;
- derivative metadata export;
- closure-table generation;
- WASM reference functions;
- optional WGSL/table-interpolation kernel generation;
- Moonlab task invocation when closure derivation requires quantum response data.

Eshkol does not own P2P networking, visualization, distributed authority, or unsupervised GPU scheduling.

2.3.3 2.3 Moonlab

Moonlab is the quantum and many-body backend. It hosts:

- state-vector tasks;
- tensor-network tasks;
- Hamiltonian matvec and Lanczos/Krylov primitives;
- response and transition-sample tasks;
- quantum chemistry / VQE-style reference jobs where available;
- CPU/WebGPU parity checks;
- ULG-compatible quantum response artifacts.

Moonlab does not own the distributed simulation state, closure cache publication, or worker-tree authority.

2.3.4 2.4 ULG carrier runtime

The ULG runtime consumes validated closures and runs scalable simulation kernels:

- carrier integration;
- spatial hash / neighborhood construction;
- edge-message evaluation;
- SPH-like smoothing observers;

- plasma/MHD/radiation kernels;
- closure interpolation;
- invariant/reduction passes;
- compact delta emission.

The ULG runtime may be implemented as a first-party PeerCompute service or as a separately registered service.

2.4 3. Core design rules

1. **PeerCompute owns orchestration.** Eshkol and Moonlab are service workers, not network authorities.
2. **Nested workers are supervised.** Eshkol/Moonlab may spawn child workers only through PeerCompute-granted leases.
3. **GPU use is brokered.** Services request WebGPU leases from PeerCompute instead of independently competing with rendering and simulation kernels.
4. **Every result is provenance-bearing.** Artifacts must identify method, input hash, source service, assumptions, validity, uncertainty, and validation status.
5. **CPU/WASM remains the reference path.** WebGPU acceleration is accepted only with tolerance policies and parity testing.
6. **Shared ABI before performance work.** The ULG GPU ABI defines dtype layout, tensor layout, complex representation, closure table format, kernel descriptors, and tolerance reports.
7. **No opaque compute islands.** Worker trees, child tasks, GPU leases, memory pressure, validation status, and closure cache state are visible to PeerCompute and NetViz.
8. **No full-state replication by default.** Distributed tasks exchange hashes, manifests, compact deltas, artifacts, closure handles, and boundary data.

2.5 4. Runtime topology

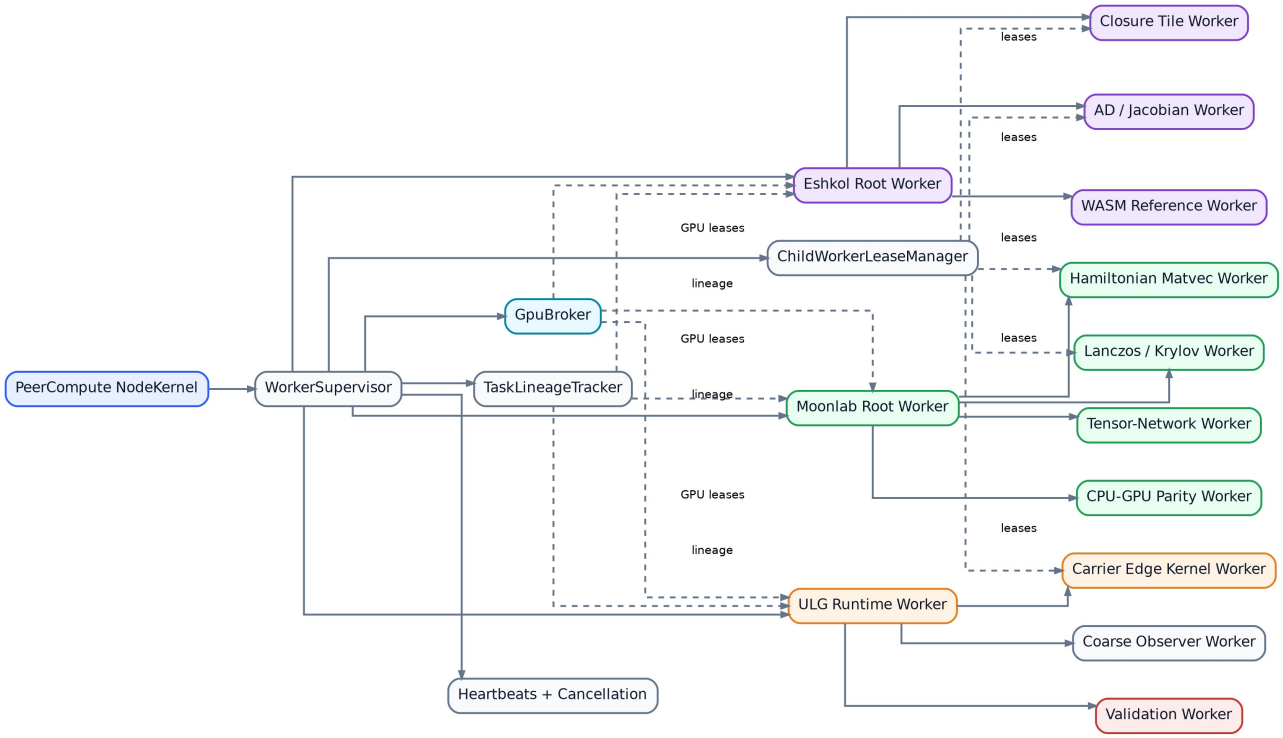


Figure 2: Supervised worker tree

2.6 5. Compute service manifest

Every service registers a manifest with PeerCompute.

```
type ComputeServiceManifest = {
  serviceId: "eshkol" | "moonlab" | "ulg-runtime" | string
  version: string
  runtime: "wasm" | "webgpu" | "wasm-webgpu" | "js" | "native-bridge"

  entry: {
    workerModule: string
    wasmModule?: string
    initExport?: string
  }

  childWorkers: {
    allowed: boolean
    maxChildren: number
    allowedModules: string[]
    sameOriginOnly: true
  }

  resources: {
    maxWasmMemoryBytes: number
    maxGpuMemoryBytes?: number
    maxCpuWorkers?: number
    maxTaskMs?: number
  }

  capabilities: string[]
  abi: {
    ulgIrVersion: string
    gpuAbiVersion: string
    supportedDTypes: string[]
    supportedLayouts: string[]
  }

  validation: {
    requiresCpuReference: boolean
    toleranceProfile: string
    parityModes: string[]
  }
}
```

2.7 6. Task capsule

All work is packaged as task capsules.

```
type ULGTaskCapsule = {
  taskId: string
  rootTaskId: string
  serviceId: string
  taskKind:
    | "closure.derive"
    | "closure.table.generate"
    | "quantum.response"
    | "quantum.spectrum"
    | "quantum.tensor_network"
    | "simulation.step"
```

```
    | "validation.reference"
    | "artifact.cache.publish"

  inputs: ArtifactRef[]
  outputs: ArtifactDecl[]
  unitsHash: string
  inputHash: string
  methodHash: string
  deterministicSeed?: string

  resources: {
    childWorkers?: number
    gpu?: "none" | "optional" | "required"
    wasmMemoryBytes?: number
    gpuMemoryBytes?: number
    priority: "render" | "interactive" | "simulation" | "background" | "validation"
  }

  validation: {
    mode: "none" | "self" | "cpu-reference" | "quorum" | "spot-check"
    toleranceProfile: string
  }

  provenance: ProvenanceBlock
}
```

2.8 7. Child-worker leases

A root service worker requests child-worker leases from PeerCompute.

```
type ChildWorkerLease = {
  leaseId: string
  parentWorkerId: string
  rootTaskId: string
  module: string
  count: number
  expiresAt?: number
  resources: {
    wasmMemoryBytes: number
    cpuWeight: number
    gpu: boolean
    expectedDurationMs?: number
  }
  cancellationToken: string
}
```

A service must not call `new Worker()` for arbitrary modules. It must use leased, allowlisted, same-origin modules that PeerCompute can observe and cancel.

2.9 8. GPU lease and shared GPU ABI

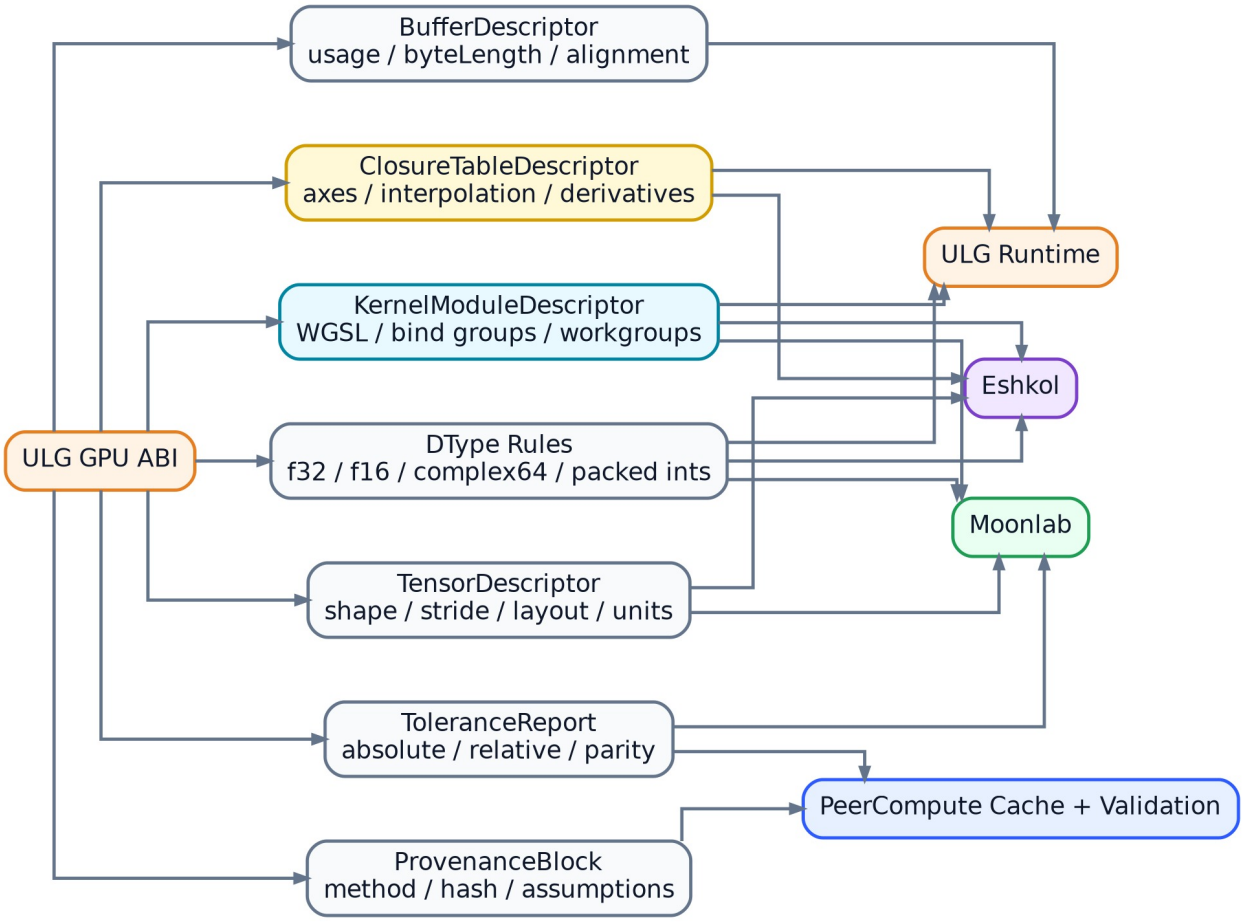


Figure 3: Shared ULG GPU ABI

```
type GpuLease = {
  leaseId: string
  serviceId: string
  rootTaskId: string
  priority: "render" | "interactive" | "simulation" | "background" | "validation"
  adapterFingerprint: string
  limitsHash: string
  maxBufferBytes: number
  maxEstimatedMemoryBytes: number
  expiresAt?: number
  cancellationToken: string
}
```

The ULG GPU ABI defines:

- scalar dtypes: `f32`, optional `f16`, packed integer types;
- complex layout: `complex64` as either interleaved `[re, im]` or SoA with descriptor flag;
- tensor descriptors: shape, stride, dtype, layout, units hash;
- closure table descriptors: axis ranges, interpolation mode, derivative availability;
- kernel descriptors: WGSL module hash, bind-group layout, workgroup size, resource class;
- tolerance reports: absolute, relative, stochastic, CPU/GPU parity, drift.

2.10 9. Artifacts

2.10.1 9.1 Quantum response artifact

```
type QuantumResponseArtifact = {
  artifactId: string
  sourceService: "moonlab"
  taskKind: "quantum.response" | "quantum.spectrum" | "quantum.tensor_network"
  inputHash: string
  method: string
  basis?: string
  representation: "state_vector" | "tensor_network" | "density_matrix" | "operator" |
↪   "samples"
  outputs: {
    energyLevels?: ArtifactRef
    groundStateEnergy?: number
    transitionMatrixElements?: ArtifactRef
    densityOfStates?: ArtifactRef
    forceSamples?: ArtifactRef
    correlationFunctions?: ArtifactRef
  }
  uncertainty: UncertaintyReport
  validation: ValidationReport
  provenance: ProvenanceBlock
}
```

2.10.2 9.2 Closure artifact

```
type ClosureArtifact = {
  closureId: string
  sourceService: "eshkol" | "imported" | "manual"
  sourceLanguage?: "eshkol"
  closureKind:
    | "eos"
    | "opacity"
    | "interatomic_potential"
    | "transport"
    | "optical_response"
    | "material_tensor"
    | "reaction_rate"

  inputs: PortDecl[]
  outputs: PortDecl[]
  derivatives?: DerivativeDecl[]

  execution: {
    mode: "wasm-reference" | "wgsl-kernel" | "table-interpolation" | "hybrid"
    wasmRef?: ArtifactRef
    wgslModule?: ArtifactRef
    table?: ArtifactRef
  }

  validity: ValidityEnvelope
  uncertainty: UncertaintyReport
  validation: ValidationReport
  provenance: ProvenanceBlock
}
```

2.11 10. Provenance block

```
type ProvenanceBlock = {
  createdAt: string
  createdByPeer: string
  sourceRepo?: string
  sourceCommit?: string
  sourceService: string
  sourceVersion: string
  method: string
  assumptions: string[]
  inputHash: string
  methodHash: string
  artifactHash: string
  parentArtifacts?: string[]
}
```

2.12 11. Validation report

```
type ValidationReport = {
  status: "unchecked" | "pass" | "warn" | "fail" | "quarantined"
  validatorPeer?: string
  validationMode: "self" | "cpu-reference" | "quorum" | "spot-check" | "invariant-audit"
  toleranceProfile: string
  maxAbsError?: number
  maxRelError?: number
  invariantDrift?: Record<string, number>
  notes?: string[]
}
```

2.13 12. Closure factory flow

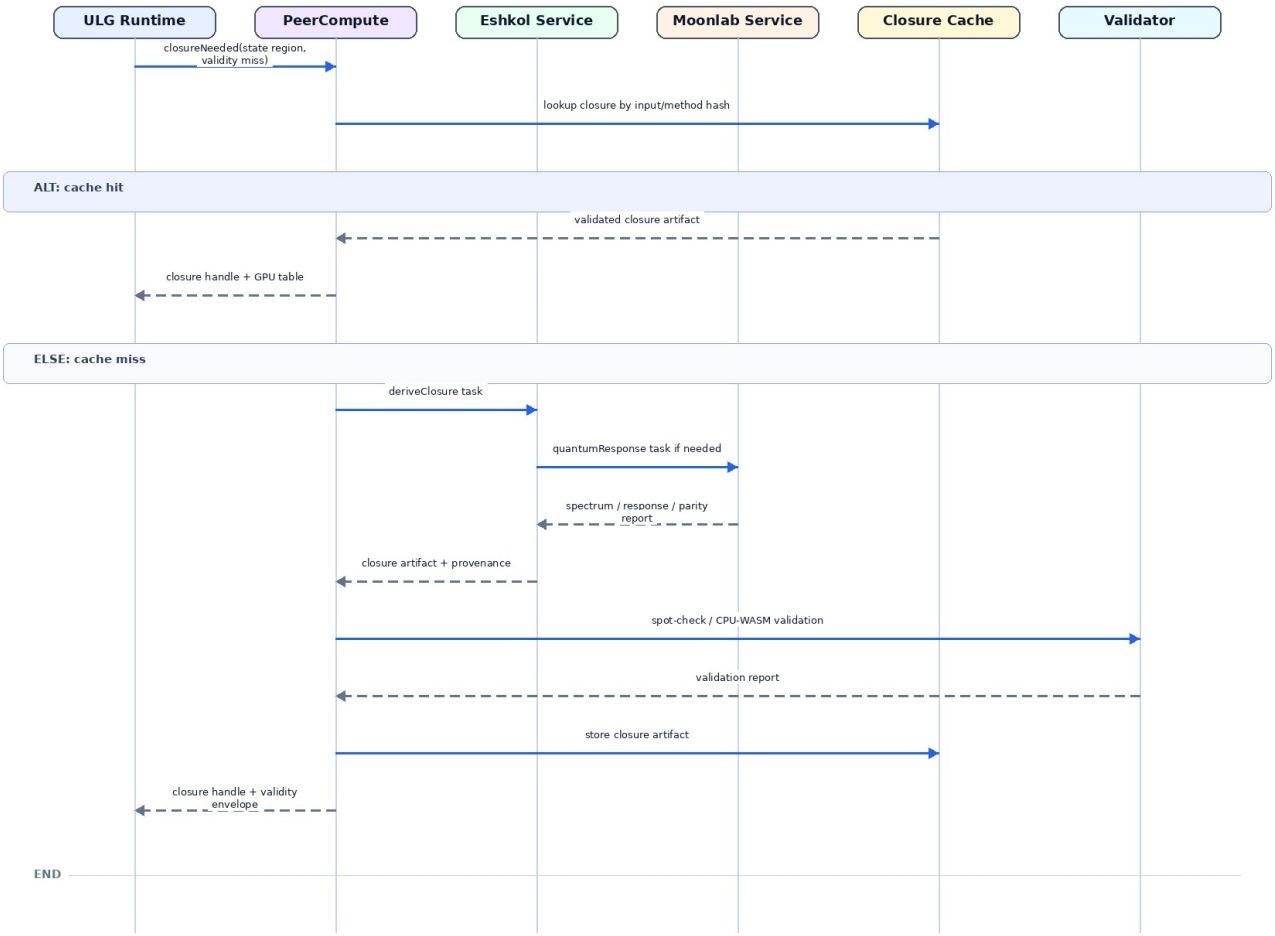


Figure 4: Closure derivation sequence

2.14 13. Security and resource model

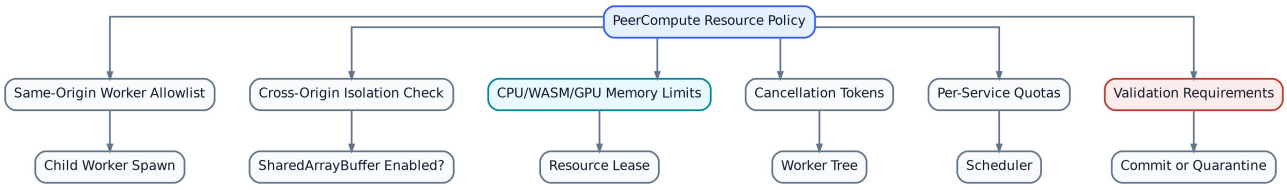


Figure 5: Resource and security policy

Requirements:

- Child worker modules must be same-origin and allowlisted.
- `SharedArrayBuffer` may be used only when cross-origin isolation is available.
- Large data should move through transferables, shared memory, IndexedDB artifact refs, or GPU buffers; avoid large structured clones.
- PeerCompute must be able to cancel a whole worker tree.

- WebGPU device-lost events must cause task retry, fallback, or quarantine.
- No service may publish distributed state directly. State commits go through PeerCompute.

2.15 14. Conformance tests

A v0.5-conforming prototype should pass:

1. Register Eshkol, Moonlab, and ULG services with manifests.
2. Spawn root service workers under PeerCompute.
3. Grant and revoke child-worker leases.
4. Grant and revoke GPU leases.
5. Run a Moonlab CPU/WebGPU parity task and emit a `QuantumResponseArtifact`.
6. Run an Eshkol closure derivation task and emit a `ClosureArtifact`.
7. Cache an artifact by input/method/artifact hash.
8. Use a cached closure in a ULG WebGPU kernel.
9. Show worker tree, GPU lease, cache, and validation telemetry in NetViz.
10. Cancel a root task and confirm all children stop.

2.16 Source notes

This specification is based on the public project documentation checked on 2026-06-05:

- PeerCompute repository and README
- Eshkol repository and README
- Moonlab repository and README
- Moonlab architecture notes
- Moonlab WebGPU plan
- Web Workers documentation
- Worker WebGPU entry point
- SharedArrayBuffer isolation requirements
- WGSL specification

3 Part 2 — Architecture Plan

Version: **0.5**
Date: **2026-06-05**

3.1 Abstract

This v0.5 specification defines a combined browser-native scientific-computing stack built from **PeerCompute**, **Eshkol**, **Moonlab**, and a Universal Law Graph (**ULG**) simulation runtime. The goal is to let distributed browser peers run multiscale physics workloads where low-level quantum and mathematical tasks generate validated closures that are consumed by scalable WebGPU simulation kernels.

For readers unfamiliar with the three libraries: **PeerCompute** is treated as the browser-based P2P orchestration layer. It owns session topology, distributed task scheduling, shared hot/warm/cold state, compute-worker supervision, validation, artifact caching, and visualization. **Eshkol** is treated as a Scheme-like mathematical/law language for automatic differentiation, free-energy and response-function authoring, closure derivation, and WASM/WebGPU-compatible numerical compilation. **Moonlab** is treated as the quantum and many-body solver backend: it produces spectra, response samples, state-vector/tensor-network results, Hamiltonian evaluations, and CPU/WebGPU parity reports used by Eshkol and ULG. In this design, **PeerCompute is the authority**, while **Eshkol and Moonlab run as supervised WASM/WebGPU compute services** that may spawn child workers only through PeerCompute leases.

The plan assumes all three libraries will be extended. PeerCompute gains service registration, worker-tree supervision, GPU/resource leasing, closure/artifact registries, and NetViz extensions. Eshkol gains ULG law macros, closure manifests, derivative metadata, Moonlab task invocation, WGSL/table-generation targets, and WASM reference exports. Moonlab gains ULG quantum-task APIs, response-artifact schemas, WebGPU kernel expansion, and deterministic CPU/GPU parity reporting. A shared **ULG GPU ABI** connects all three so tensors, complex numbers, closure tables, worker messages, tolerance reports, and provenance blocks have one common representation.

3.2 1. Architectural thesis

The combined system should be built as a **service-oriented browser scientific runtime**:

```
PeerCompute  = authority, network, state, visualization, supervision
Eshkol       = mathematical law compiler and closure factory
Moonlab      = quantum/many-body response engine
ULG Runtime  = scalable carrier simulation consuming validated closures
WebGPU/WGSL  = hot dense numerical kernels
WASM/CPU     = reference and fallback execution
```

The key architecture choice is that Moonlab and Eshkol run inside PeerCompute's supervised compute tree. They may spawn child workers, but only through PeerCompute leases. This avoids opaque compute islands and lets the same orchestration layer handle distribution, visualization, validation, cancellation, cache publication, and resource pressure.

3.3 2. System context

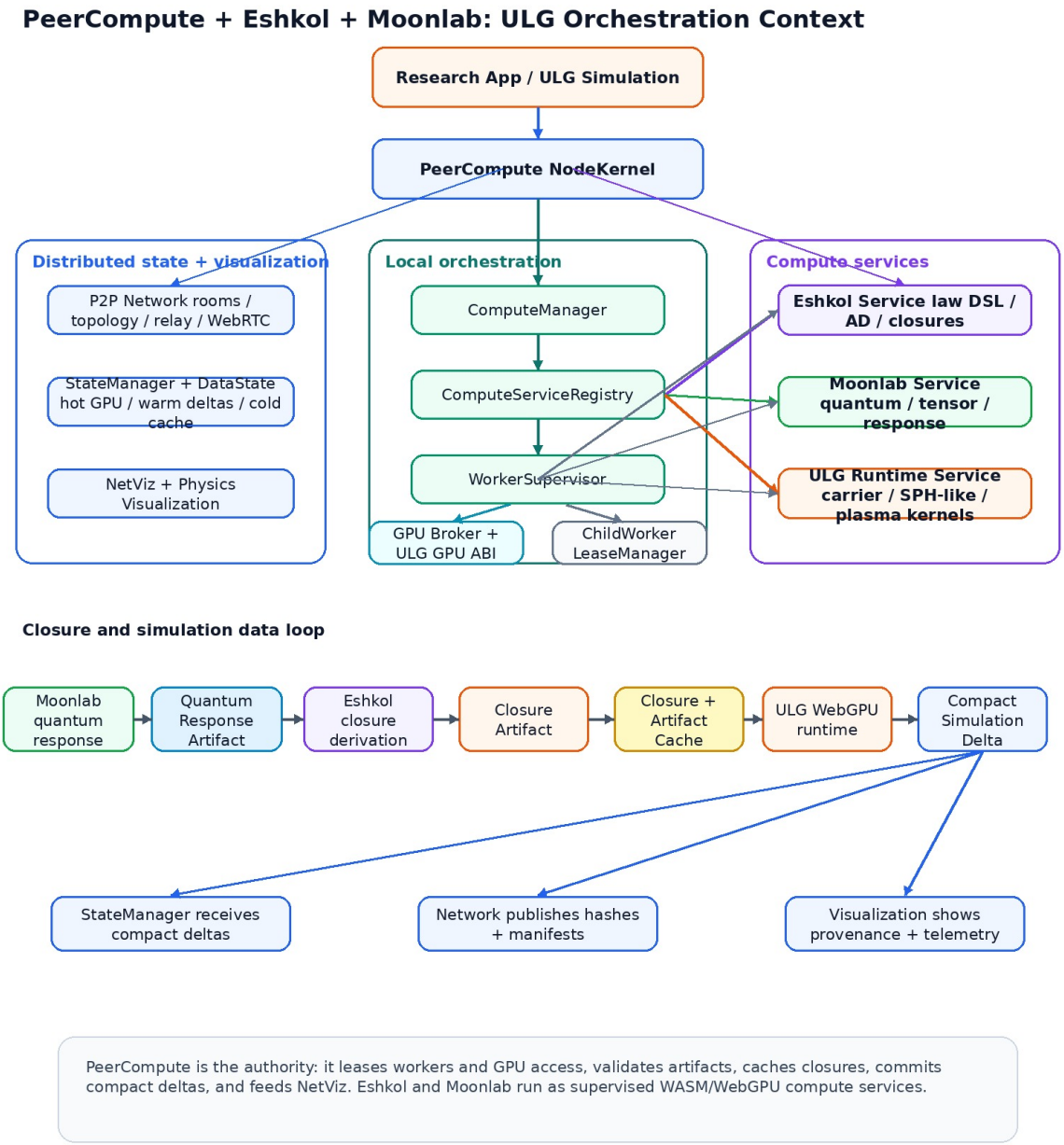


Figure 6: System context and orchestration loop

3.4 3. Logical layers

3.4.1 Layer A — PeerCompute orchestration

Responsibilities:

- peer discovery, rooms, relay/direct topology, and network state;
- distributed task scheduling;
- service registration and capability discovery;
- worker supervision and cancellation;
- GPU/resource leasing;

- hot/warm/cold data lifecycle;
- artifact cache and closure registry;
- validation/quorum/spot-checks;
- NetViz and physics telemetry.

3.4.2 Layer B — Compute services

Services registered under PeerCompute:

- `eshkol`: law DSL, AD, closure derivation, table generation, WASM/WGSL outputs.
- `moonlab`: state-vector/tensor-network/quantum response tasks.
- `ulg-runtime`: carrier, SPH-like, MHD, radiation, observer, and validation kernels.

3.4.3 Layer C — Shared ABI and artifacts

All services communicate through:

- ULG IR manifests;
- ULG GPU ABI;
- typed artifacts;
- validation reports;
- provenance blocks;
- closure cache records;
- task lineage metadata.

3.4.4 Layer D — Hot execution

Hot execution lives in WebGPU workers and WASM:

- WGSL compute kernels;
- WASM reference functions;
- child workers for parallel CPU/WASM tasks;
- optional shared-memory paths when available.

3.5 4. Worker-tree architecture

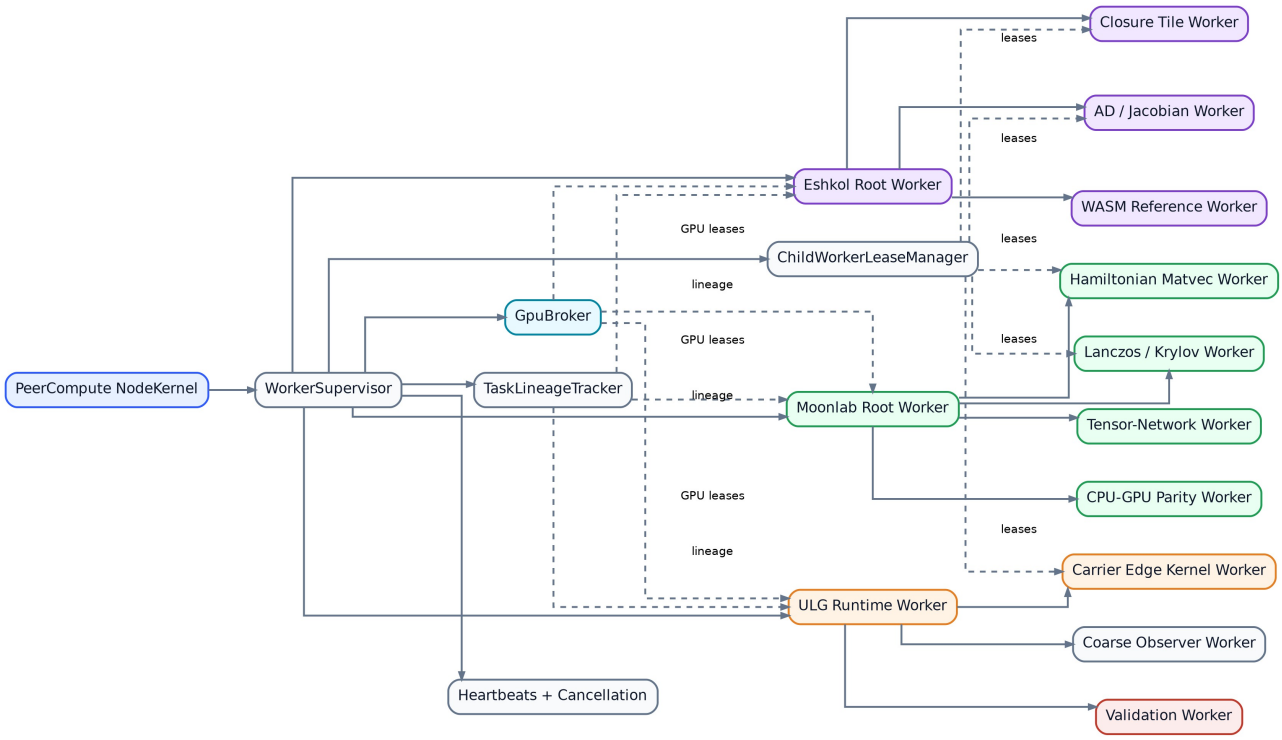


Figure 7: Supervised worker tree

3.5.1 Rules

1. Root workers are spawned by PeerCompute.
2. Child workers are spawned only with PeerCompute leases.
3. Each worker reports heartbeat, task status, memory estimates, GPU lease usage, and child status.
4. Parent service workers aggregate child results before returning to PeerCompute.
5. PeerCompute can cancel a root task and its entire subtree.

3.6 5. PeerCompute component additions

3.6.1 5.1 `ComputeServiceRegistry`

Loads service manifests and exposes capabilities.

```
interface ComputeServiceRegistry {
  register(manifest: ComputeServiceManifest): Promise<ServiceHandle>
  listCapabilities(): ServiceCapabilityIndex
  resolve(taskKind: string): ServiceHandle[]
}
```

3.6.2 5.2 `WorkerSupervisor`

Manages root workers, child workers, lifetimes, cancellation, health, and telemetry.

```
interface WorkerSupervisor {
  spawnService(serviceId: string): Promise<RootWorkerHandle>
  submitTask(task: ULGTaskCapsule): Promise<ComputeResult>
  cancelTree(rootTaskId: string): Promise<void>
```

```
  getTreeTelemetry(rootTaskId: string): WorkerTreeTelemetry
}
```

3.6.3 5.3 ChildWorkerLeaseManager

Prevents runaway nested workers.

```
interface ChildWorkerLeaseManager {
  request(parentWorkerId: string, spec: ChildWorkerLeaseRequest):
   ↪  Promise<ChildWorkerLease>
  release(leaseId: string): Promise<void>
  revokeByRootTask(rootTaskId: string): Promise<void>
}
```

3.6.4 5.4 GpuBroker

Brokers WebGPU access across rendering, ULG simulation, Moonlab, and Eshkol.

```
interface GpuBroker {
  probe(): Promise<GpuCapabilities>
  requestLease(spec: GpuLeaseRequest): Promise<GpuLease>
  releaseLease(leaseId: string): Promise<void>
  reportPressure(): GpuPressureReport
}
```

Priority order should be:

```
render / UI > interactive simulation > simulation step > closure generation > quantum
 ↪  reference > background validation
```

3.6.5 5.5 ArtifactCache and ClosureRegistry

Artifacts are content-addressed by input hash, method hash, and artifact hash.

```
interface ArtifactCache {
  put(artifact: ArtifactRecord): Promise<ArtifactRef>
  get(ref: ArtifactRef): Promise<ArtifactRecord | undefined>
  announce(ref: ArtifactRef): Promise<void>
}

interface ClosureRegistry {
  resolve(query: ClosureQuery): Promise<ClosureArtifact | undefined>
  store(closure: ClosureArtifact): Promise<ArtifactRef>
  invalidate(reason: InvalidationEvent): Promise<void>
}
```

3.7 6. Eshkol service architecture

```
Eshkol Root Worker
  ├─ parser/macro expansion
  ├─ type/unit/dimensional metadata
  ├─ AD engine
  ├─ closure table generator
  ├─ WASM reference export
  ├─ WGSL/table strategy emitter
  ├─ Moonlab task client
```

```
  └─ child workers for tile generation and validation
```

Eshkol should expose service task kinds:

```
eshkol.law.compile
eshkol.closure.derive
eshkol.closure.table.generate
eshkol.derivative.export
eshkol.wasm.reference.build
eshkol.wgsl.emit
eshkol.validation.reference
```

3.8 7. Moonlab service architecture

```
Moonlab Root Worker
  ├─ WASM module loader
  ├─ WebGPU backend probe
  ├─ state-vector task adapter
  ├─ tensor-network task adapter
  ├─ Hamiltonian/matvec task adapter
  ├─ Lanczos/Krylov task adapter
  ├─ response artifact exporter
  └─ CPU/GPU parity runner
```

Moonlab should expose service task kinds:

```
moonlab.quantum.response
moonlab.quantum.spectrum
moonlab.quantum.ground_state
moonlab.quantum.transition_matrix
moonlab.quantum.correlation
moonlab.tensor_network.contract
moonlab.tensor_network.expectation
moonlab.parity.cpu_webgpu
```

3.9 8. Shared GPU ABI

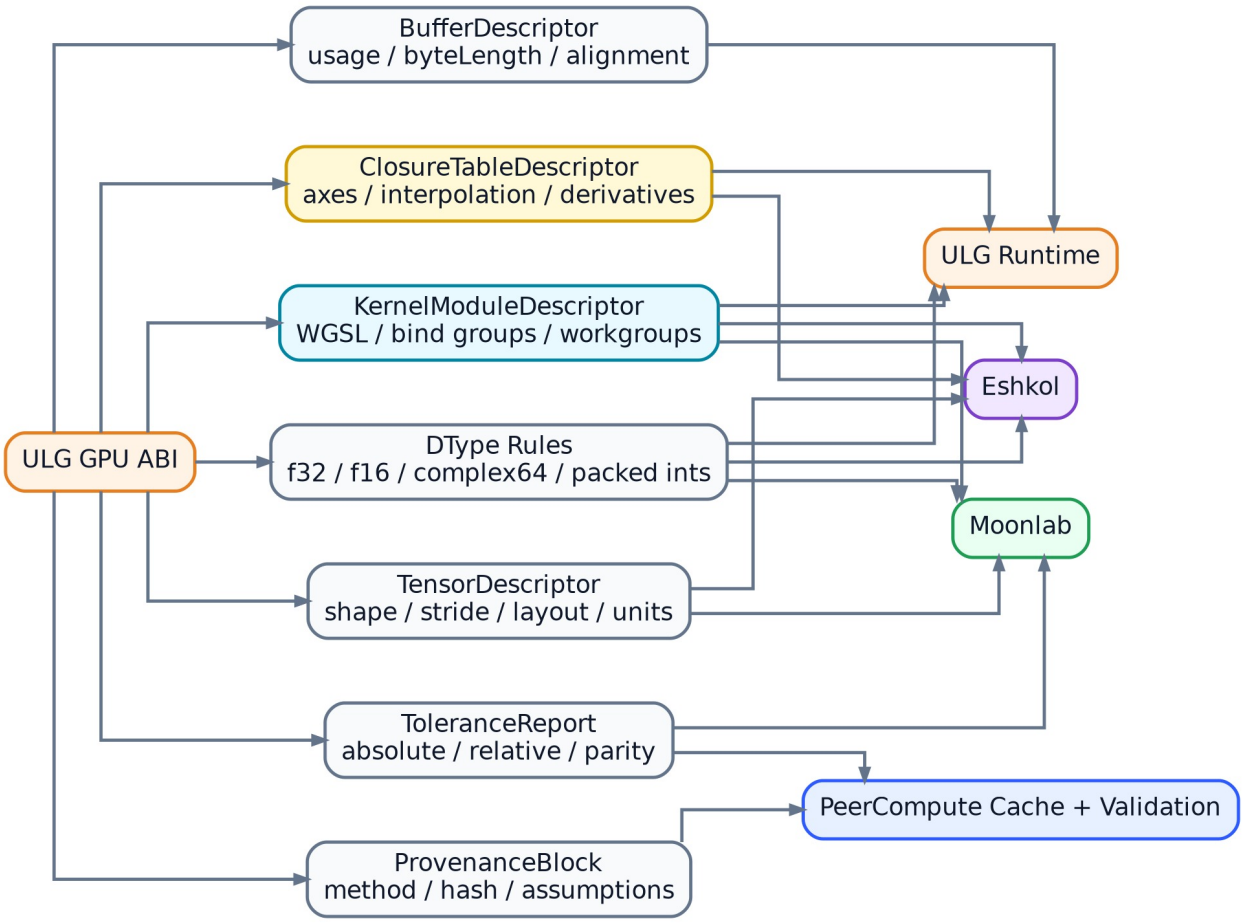


Figure 8: Shared ULG GPU ABI

The ABI should be packaged as a small shared module, not copied independently into all three repos. Recommended package boundary:

```
ulg-gpu-abi/
  src/types.ts
  src/schemas/
  src/wgsl/
  src/layouts/
  src/validation/
  conformance/
```

3.10 9. Data lifecycle

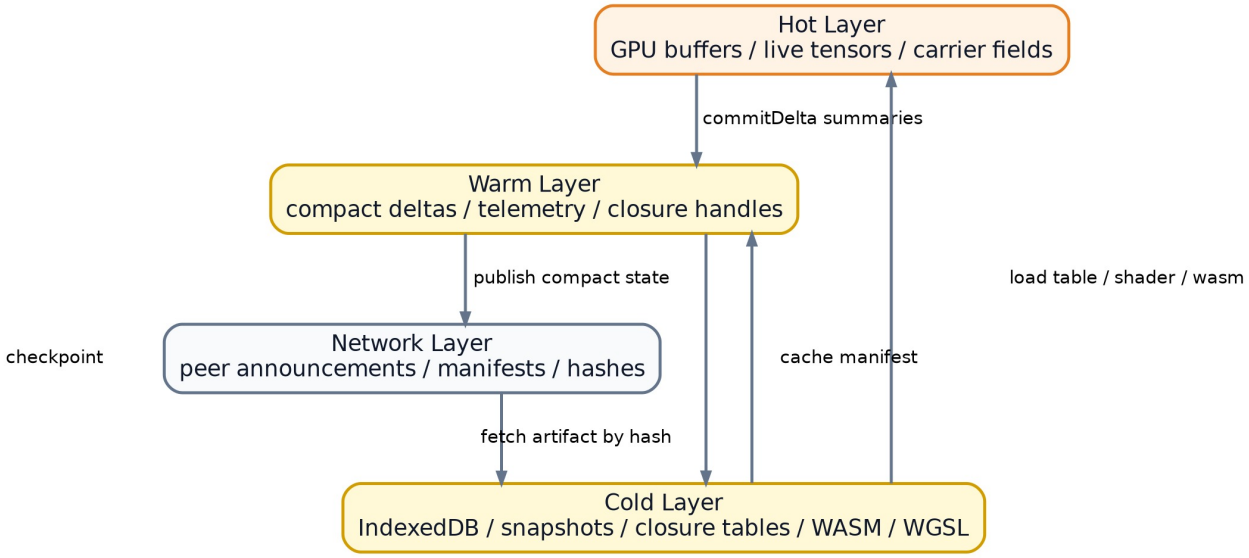


Figure 9: Data lifecycle across hot, warm, cold, and network layers

3.10.1 Hot layer

- GPU buffers;
- live tensors;
- carrier fields;
- temporary reduction buffers;
- simulation domain state.

3.10.2 Warm layer

- compact deltas;
- telemetry;
- closure handles;
- task status;
- peer announcements.

3.10.3 Cold layer

- IndexedDB snapshots;
- cached closure tables;
- WASM modules;
- WGSL modules;
- quantum response artifacts;
- validation reports.

3.11 10. End-to-end execution flows

3.11.1 10.1 Closure derivation flow

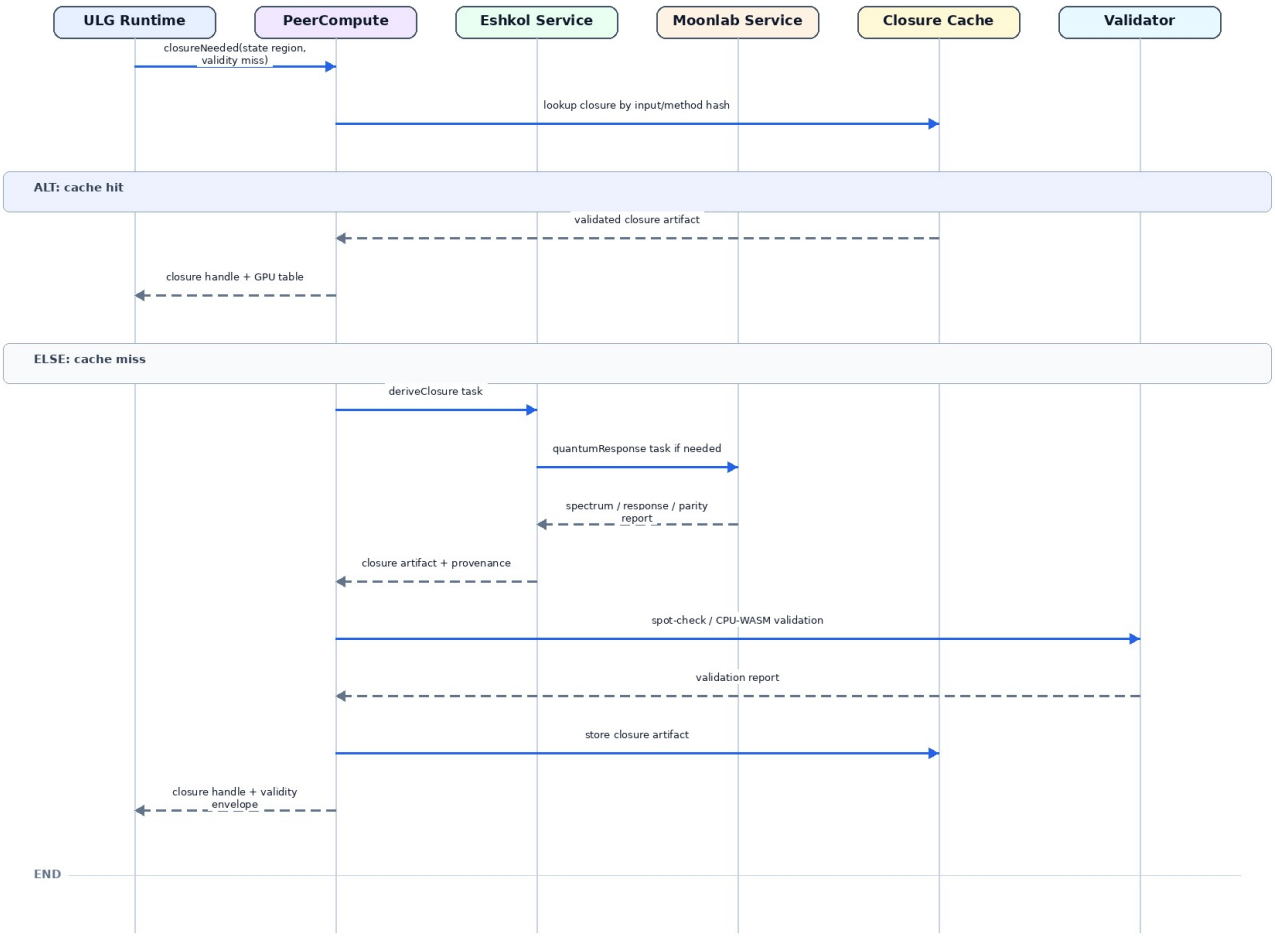


Figure 10: Closure derivation sequence

3.11.2 10.2 ULG simulation flow

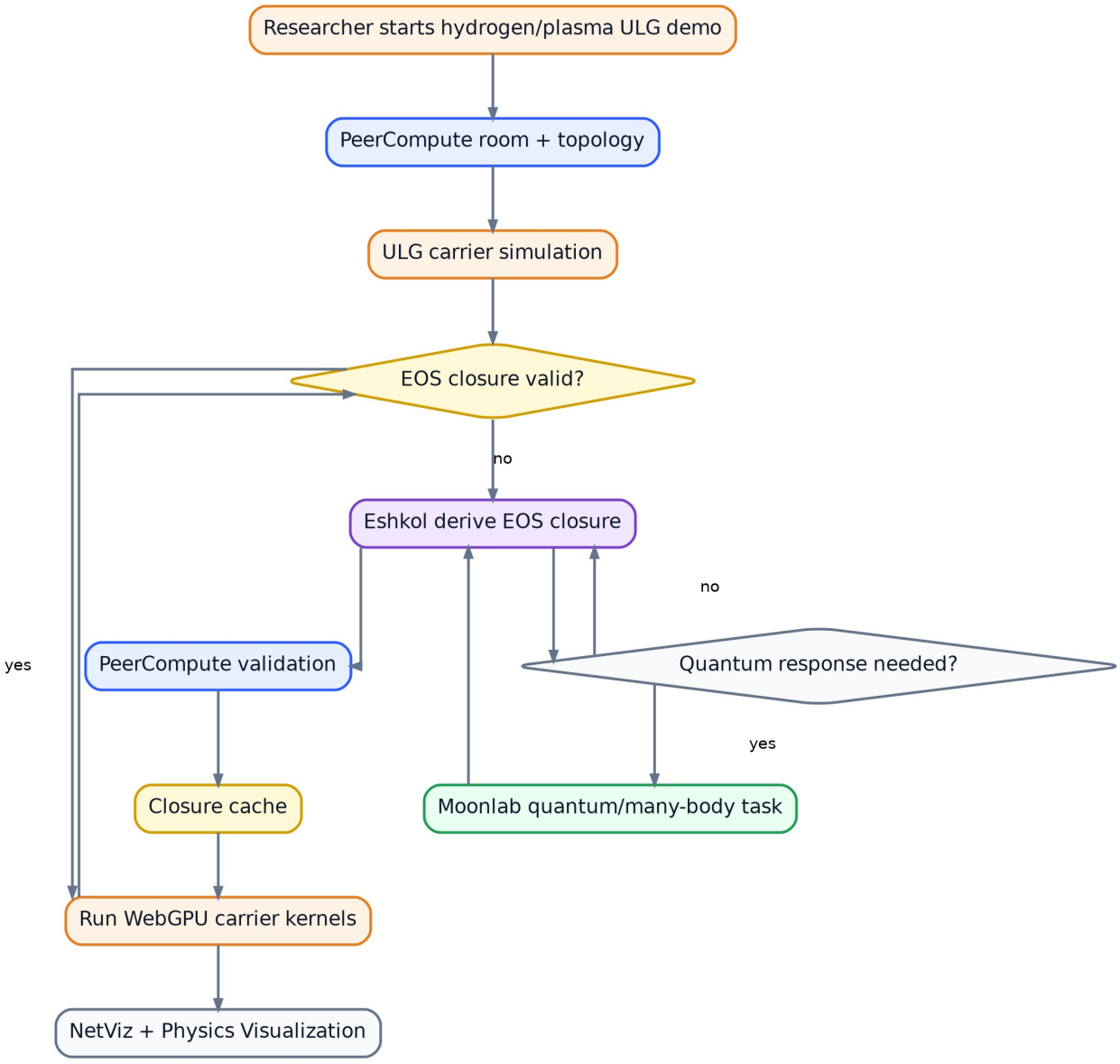


Figure 11: Hydrogen plasma demo flow

3.11.3 10.3 Task lifecycle

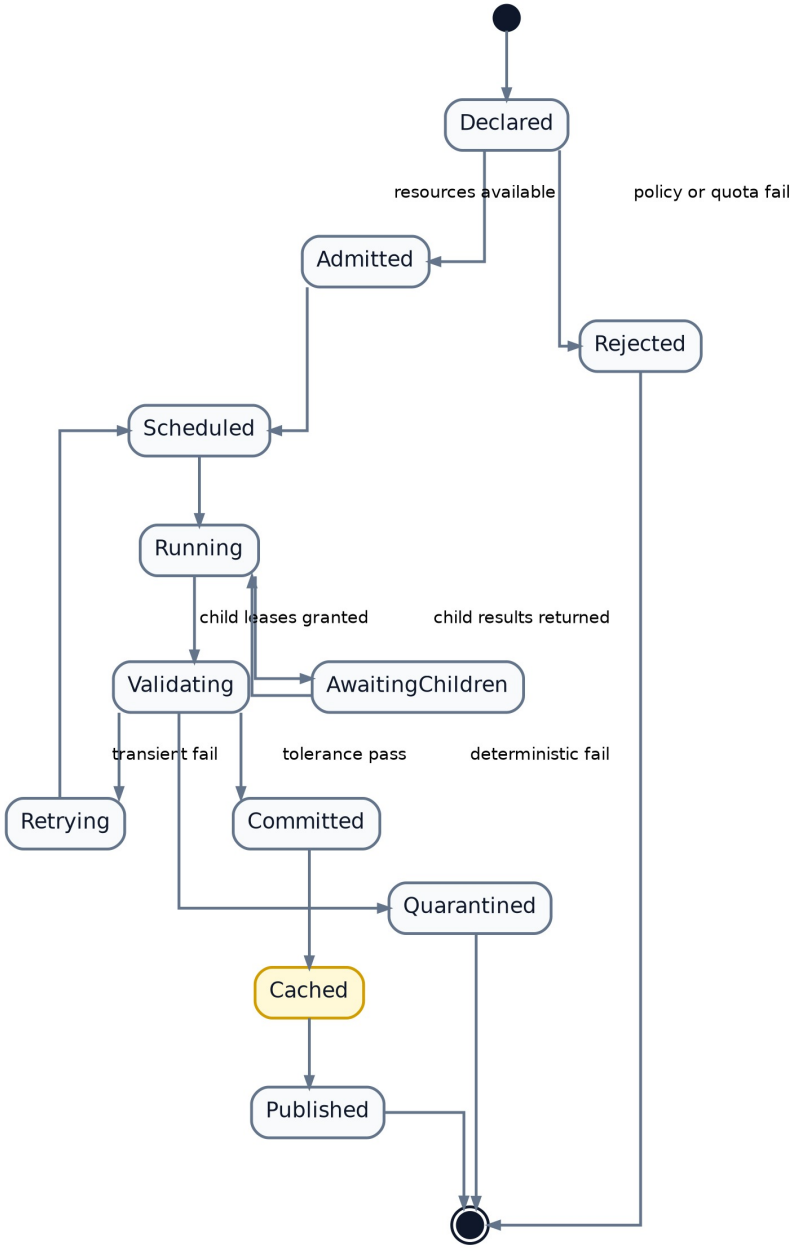


Figure 12: Task lifecycle state machine

3.12 11. Validation and provenance

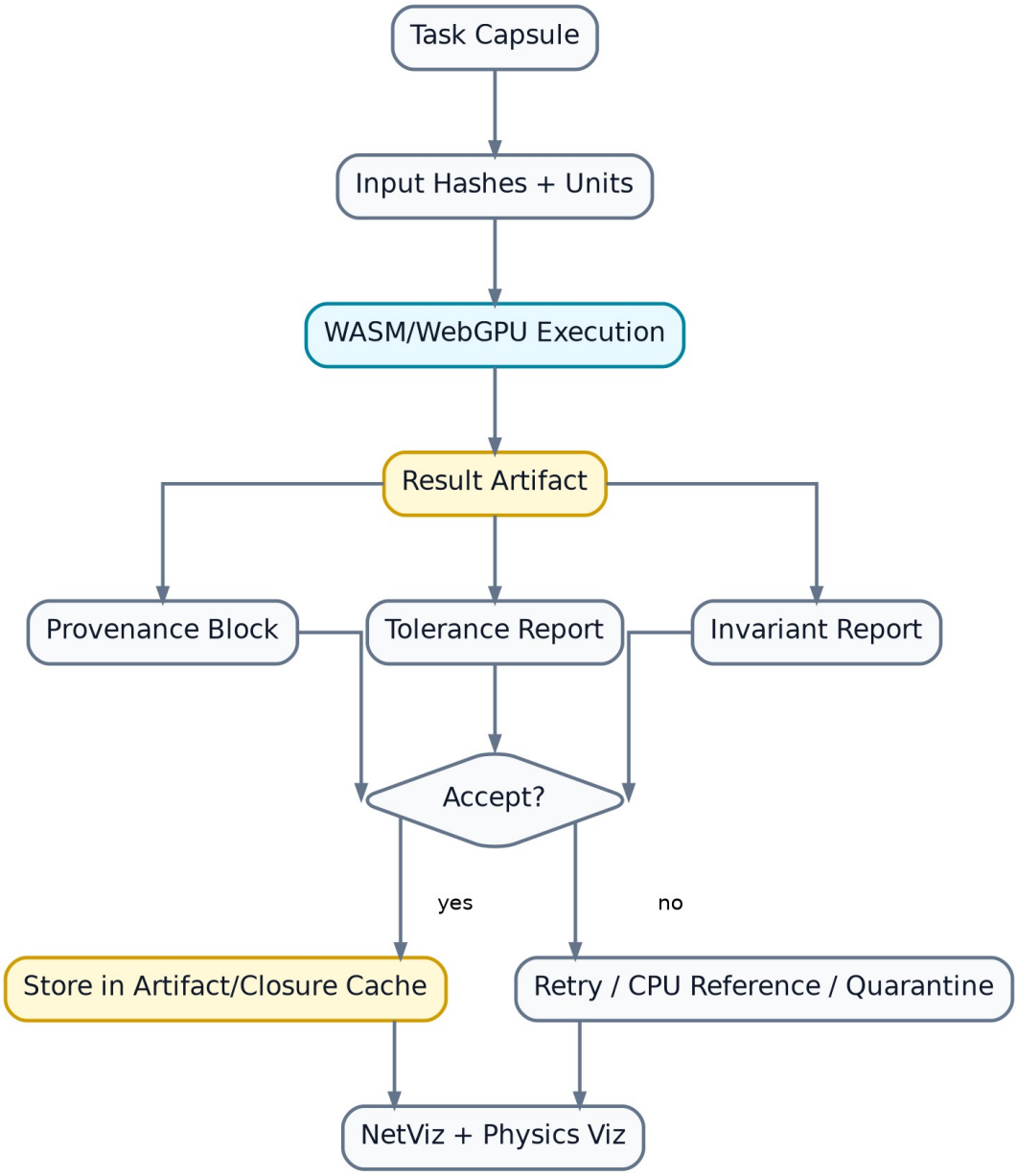


Figure 13: Validation and provenance loop

Validation layers:

1. local self-check in the service;
2. CPU/WASM reference comparison;
3. randomized spot-checks;
4. peer quorum for expensive results;
5. physics invariant audit;
6. cache quarantine on deterministic failure.

3.13 12. Visualization architecture

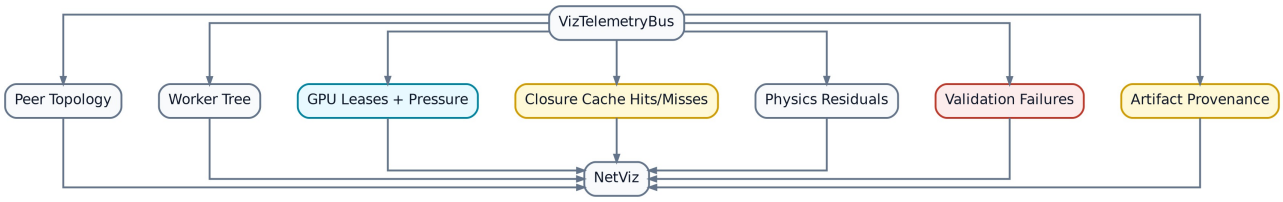


Figure 14: Visualization telemetry bus

NetViz should gain panels for:

- service registry and capabilities;
- worker tree;
- child-worker leases;
- GPU pressure;
- artifact cache;
- closure provenance;
- validation failures;
- physics residuals;
- simulation domain ownership.

3.14 13. Architecture risks

Risk	Mitigation
Eshkol and Moonlab invent incompatible GPU layouts	Shared ULG GPU ABI first
Child workers run away	Lease manager, quotas, cancellation tree
GPU compute starves visualization	GPU broker priority classes
Closure cache stores bad science	provenance + validation + quarantine
WebGPU parity differs by adapter	tolerance profiles + CPU/WASM reference
Large tensors are cloned repeatedly	transferables, SharedArrayBuffer when isolated, artifact refs
Peer network is flooded with raw state	compact deltas and content-addressed artifacts

3.15 Source notes

This specification is based on the public project documentation checked on 2026-06-05:

- PeerCompute repository and README
- Eshkol repository and README
- Moonlab repository and README
- Moonlab architecture notes
- Moonlab WebGPU plan
- Web Workers documentation
- Worker WebGPU entry point
- SharedArrayBuffer isolation requirements
- WGSL specification

4 Part 3 — Implementation Roadmap

Version: **0.5**
Date: **2026-06-05**

4.1 Abstract

This v0.5 specification defines a combined browser-native scientific-computing stack built from **PeerCompute**, **Eshkol**, **Moonlab**, and a Universal Law Graph (**ULG**) simulation runtime. The goal is to let distributed browser peers run multiscale physics workloads where low-level quantum and mathematical tasks generate validated closures that are consumed by scalable WebGPU simulation kernels.

For readers unfamiliar with the three libraries: **PeerCompute** is treated as the browser-based P2P orchestration layer. It owns session topology, distributed task scheduling, shared hot/warm/cold state, compute-worker supervision, validation, artifact caching, and visualization. **Eshkol** is treated as a Scheme-like mathematical/law language for automatic differentiation, free-energy and response-function authoring, closure derivation, and WASM/WebGPU-compatible numerical compilation. **Moonlab** is treated as the quantum and many-body solver backend: it produces spectra, response samples, state-vector/tensor-network results, Hamiltonian evaluations, and CPU/WebGPU parity reports used by Eshkol and ULG. In this design, **PeerCompute is the authority**, while **Eshkol and Moonlab run as supervised WASM/WebGPU compute services** that may spawn child workers only through PeerCompute leases.

The plan assumes all three libraries will be extended. PeerCompute gains service registration, worker-tree supervision, GPU/resource leasing, closure/artifact registries, and NetViz extensions. Eshkol gains ULG law macros, closure manifests, derivative metadata, Moonlab task invocation, WGSL/table-generation targets, and WASM reference exports. Moonlab gains ULG quantum-task APIs, response-artifact schemas, WebGPU kernel expansion, and deterministic CPU/GPU parity reporting. A shared **ULG GPU ABI** connects all three so tensors, complex numbers, closure tables, worker messages, tolerance reports, and provenance blocks have one common representation.

4.2 1. Roadmap philosophy

The implementation should not start by building a full multiscale physics engine. It should start by building the **contracts and orchestration substrate** that make the physics stack credible:

```
shared ABI -> supervised workers -> closure artifacts -> quantum artifacts -> validation
 ↪  -> end-to-end demo
```

Every milestone should leave the system runnable, testable, and observable.

4.3 2. Roadmap overview

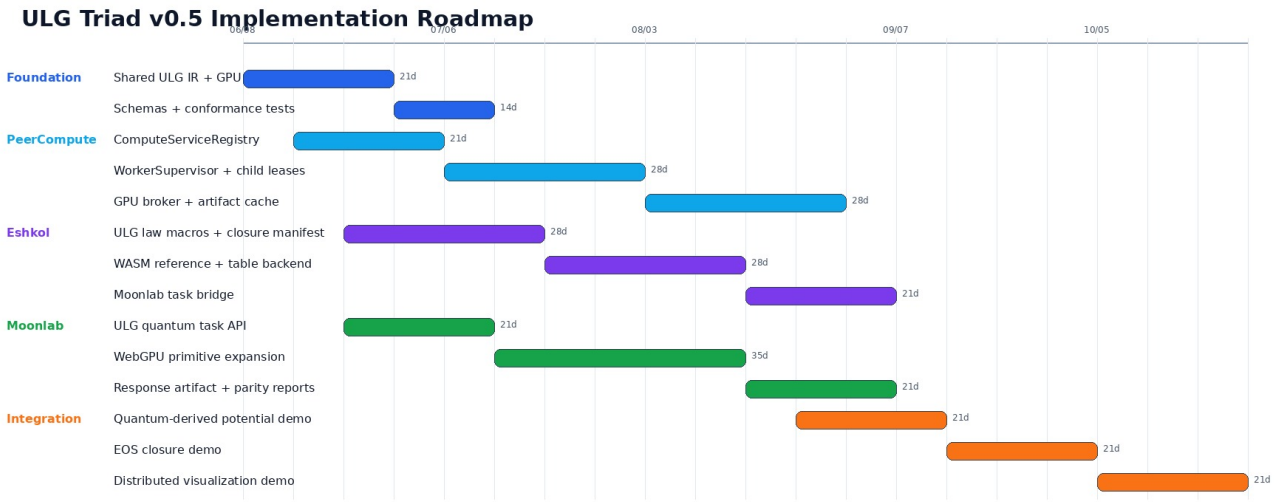


Figure 15: ULG Triad v0.5 Implementation Roadmap

4.4 3. Milestone 0.5 — specification package

Status: this document.

Deliverables:

- versioned integrated system spec;
- architecture plan;
- implementation roadmap;
- library extension plan;
- Mermaid diagrams;
- schema sketches;
- abstract for unfamiliar readers.

Acceptance:

- all documents versioned v0.5;
- all major docs include the abstract;
- diagrams compile as standalone Mermaid source;
- extension points are explicit for all three libraries.

4.5 4. Milestone 0.6 — shared ULG IR and GPU ABI

Goal: create the common contract before modifying runtime behavior deeply.

Deliverables:

- `ulg-gpu-abi` package;
- TypeScript type definitions;
- JSON schemas;
- WGSL layout snippets;
- complex64 and tensor descriptor tests;
- closure table descriptor tests;
- tolerance report schema;
- provenance block schema.

Acceptance tests:

1. Eshkol, Moonlab, and a dummy ULG worker can all import the same ABI package.

2. A complex64 vector buffer round-trips through JS/WASM/WGSL layout tests.
3. A closure table descriptor validates against schema.
4. A tolerance report can be emitted and parsed by PeerCompute.

4.6 5. Milestone 0.7 — PeerCompute service orchestration

Goal: make PeerCompute a supervised service host.

PeerCompute work:

- `ComputeServiceRegistry`;
- `WorkerSupervisor`;
- root service worker lifecycle;
- child-worker lease manager;
- task lineage tracker;
- cancellation tree;
- service heartbeat protocol;
- minimal NetViz worker-tree panel.

Acceptance tests:

1. Register a dummy Eshkol service.
2. Register a dummy Moonlab service.
3. Submit tasks to each service.
4. Grant child-worker leases.
5. Cancel a root task and verify all child workers stop.
6. Show worker tree in NetViz.

4.7 6. Milestone 0.8 — PeerCompute GPU broker and artifact cache

Goal: broker local GPU access and store content-addressed artifacts.

PeerCompute work:

- `GpuBroker` capability probing;
- GPU lease request/release;
- priority classes;
- device-lost recovery path;
- `ArtifactCache`;
- `ClosureRegistry`;
- IndexedDB storage backend;
- cache announcement protocol.

Acceptance tests:

1. Service requests optional GPU lease and receives one if available.
2. Render-priority lease preempts background compute lease.
3. Device-lost event marks running tasks retryable.
4. Artifact cache stores and retrieves by content hash.
5. Closure registry invalidates a closure when validity conditions fail.

4.8 7. Milestone 0.9 — Eshkol closure service

Goal: run Eshkol as a supervised closure compiler.

Eshkol work:

- service worker bootstrap;
- ULG law macro prototype;
- closure manifest emitter;
- derivative metadata export;
- WASM reference function export;
- closure table generation over tiles;

- child-worker tiling;
- optional WGSL/table strategy descriptor;
- PeerCompute task protocol adapter.

Acceptance tests:

1. Eshkol service compiles a simple free-energy closure.
2. Eshkol emits `ClosureArtifact` with validity and provenance.
3. Eshkol generates a closure table tile using child workers.
4. Eshkol emits derivative metadata for pressure/entropy-style outputs.
5. PeerCompute validates random table samples against WASM reference.

4.9 8. Milestone 1.0 — Moonlab quantum service

Goal: run Moonlab as a supervised quantum/many-body response backend.

Moonlab work:

- service worker bootstrap;
- ULG quantum task API;
- WebGPU capability probe;
- CPU fallback path;
- response artifact exporter;
- parity report emitter;
- WebGPU primitive expansion for complex vector ops, reductions, sparse matvec, and tensor-network kernels;
- PeerCompute task protocol adapter.

Acceptance tests:

1. Moonlab service runs a small CPU reference quantum task.
2. Moonlab runs an equivalent WebGPU task when available.
3. Moonlab emits `QuantumResponseArtifact`.
4. Moonlab emits CPU/GPU parity report.
5. Failed parity quarantines the artifact instead of caching it as valid.

4.10 9. Milestone 1.1 — Moonlab → Eshkol closure pipeline

Goal: connect quantum response artifacts to closure derivation.

End-to-end demo 1: quantum-derived bond potential.

Flow:

```
Moonlab computes energy samples -> Eshkol fits/interpolates E(r) -> Eshkol differentiates
 ↪  force -> ULG worker consumes closure table
```

Acceptance tests:

1. Force equals `-dE/dr` within tolerance.
2. Closure invalidates outside sampled range.
3. CPU/WASM reference and WGSL interpolation agree within tolerance.
4. PeerCompute displays artifact provenance.

4.11 10. Milestone 1.2 — EOS closure pipeline

Goal: demonstrate ULG's material/plasma closure factory.

End-to-end demo 2: hydrogen/plasma EOS closure.

Flow:

```
Eshkol defines free-energy model -> derives pressure/entropy/heat-capacity/ionization
 ↪  outputs -> PeerCompute validates -> ULG carrier kernel interpolates closure
```

Acceptance tests:

1. Closure artifact includes inputs `rho`, `T`, `species`.
2. Outputs include `P`, `S`, `Cv`, and ionization-like variables.
3. Derivatives `dP/drho` and `dP/dT` are available or explicitly unavailable.
4. Carrier simulation can use the closure without pulling full Eshkol into the hot kernel.

4.12 11. Milestone 1.3 — distributed visualization and telemetry

Goal: make the system debuggable.

PeerCompute/NetViz work:

- worker-tree panel;
- GPU pressure panel;
- closure cache panel;
- validation/provenance panel;
- task lineage panel;
- physics residual overlay;
- domain ownership view.

Acceptance tests:

1. NetViz shows root workers and child workers.
2. NetViz shows GPU leases by service.
3. NetViz shows closure cache hits/misses.
4. NetViz can trace a ULG closure from simulation use back to Eshkol and Moonlab artifacts.

4.13 12. Milestone 1.4 — hardening

Work:

- fuzz schemas;
- soak-test worker leaks;
- stress-test child-worker quotas;
- simulate WebGPU device loss;
- simulate peer churn;
- validate cache quarantine behavior;
- benchmark table interpolation;
- benchmark Moonlab WebGPU primitives;
- benchmark Eshkol closure generation.

Acceptance:

- no runaway worker trees;
- no untracked GPU allocations;
- no uncancellable long-running tasks;
- no artifact accepted without validation status;
- degraded browsers fall back to WASM/CPU where possible.

4.14 13. First three demos

4.14.1 Demo A — Supervised service smoke test

```
PeerCompute starts Eshkol + Moonlab services -> both spawn child workers -> both report
 ↪  telemetry -> PeerCompute cancels both cleanly
```

4.14.2 Demo B — Quantum-derived potential

```
Moonlab energy samples -> Eshkol closure -> ULG two-particle oscillator -> NetViz
 ↪  provenance
```

4.14.3 Demo C — Hydrogen/plasma closure

```
Eshkol EOS table -> ULG carrier collapse toy model -> closure invalidation and refresh
```

4.15 14. Definition of done for v1-compatible integration

The integration is credible when:

- PeerCompute supervises all root and child workers;
- Moonlab and Eshkol never publish distributed state directly;
- all heavy artifacts are content-addressed;
- WebGPU tasks have CPU/WASM fallback or explicit unsupported status;
- closure artifacts include validity, uncertainty, and provenance;
- NetViz shows enough information to debug a failed closure or quantum task;
- the first two end-to-end demos run in a browser peer.

4.16 Source notes

This specification is based on the public project documentation checked on 2026-06-05:

- PeerCompute repository and README
- Eshkol repository and README
- Moonlab repository and README
- Moonlab architecture notes
- Moonlab WebGPU plan
- Web Workers documentation
- Worker WebGPU entry point
- SharedArrayBuffer isolation requirements
- WGSL specification

5 Part 4 — Library Extension Plan

Version: **0.5**
Date: **2026-06-05**

5.1 Abstract

This v0.5 specification defines a combined browser-native scientific-computing stack built from **PeerCompute**, **Eshkol**, **Moonlab**, and a Universal Law Graph (**ULG**) simulation runtime. The goal is to let distributed browser peers run multiscale physics workloads where low-level quantum and mathematical tasks generate validated closures that are consumed by scalable WebGPU simulation kernels.

For readers unfamiliar with the three libraries: **PeerCompute** is treated as the browser-based P2P orchestration layer. It owns session topology, distributed task scheduling, shared hot/warm/cold state, compute-worker supervision, validation, artifact caching, and visualization. **Eshkol** is treated as a Scheme-like mathematical/law language for automatic differentiation, free-energy and response-function authoring, closure derivation, and WASM/WebGPU-compatible numerical compilation. **Moonlab** is treated as the quantum and many-body solver backend: it produces spectra, response samples, state-vector/tensor-network results, Hamiltonian evaluations, and CPU/WebGPU parity reports used by Eshkol and ULG. In this design, **PeerCompute is the authority**, while **Eshkol and Moonlab run as supervised WASM/WebGPU compute services** that may spawn child workers only through PeerCompute leases.

The plan assumes all three libraries will be extended. PeerCompute gains service registration, worker-tree supervision, GPU/resource leasing, closure/artifact registries, and NetViz extensions. Eshkol gains ULG law macros, closure manifests, derivative metadata, Moonlab task invocation, WGSL/table-generation targets, and WASM reference exports. Moonlab gains ULG quantum-task APIs, response-artifact schemas, WebGPU kernel expansion, and deterministic CPU/GPU parity reporting. A shared **ULG GPU ABI** connects all three so tensors, complex numbers, closure tables, worker messages, tolerance reports, and provenance blocks have one common representation.

5.2 1. Extension strategy

All three libraries should be extended, but not equally. PeerCompute needs orchestration and supervision primitives. Eshkol needs ULG law/closure authoring and compilation outputs. Moonlab needs ULG quantum-task and response-artifact surfaces. The shared ULG GPU ABI should be built before the libraries grow separate incompatible GPU data layouts.

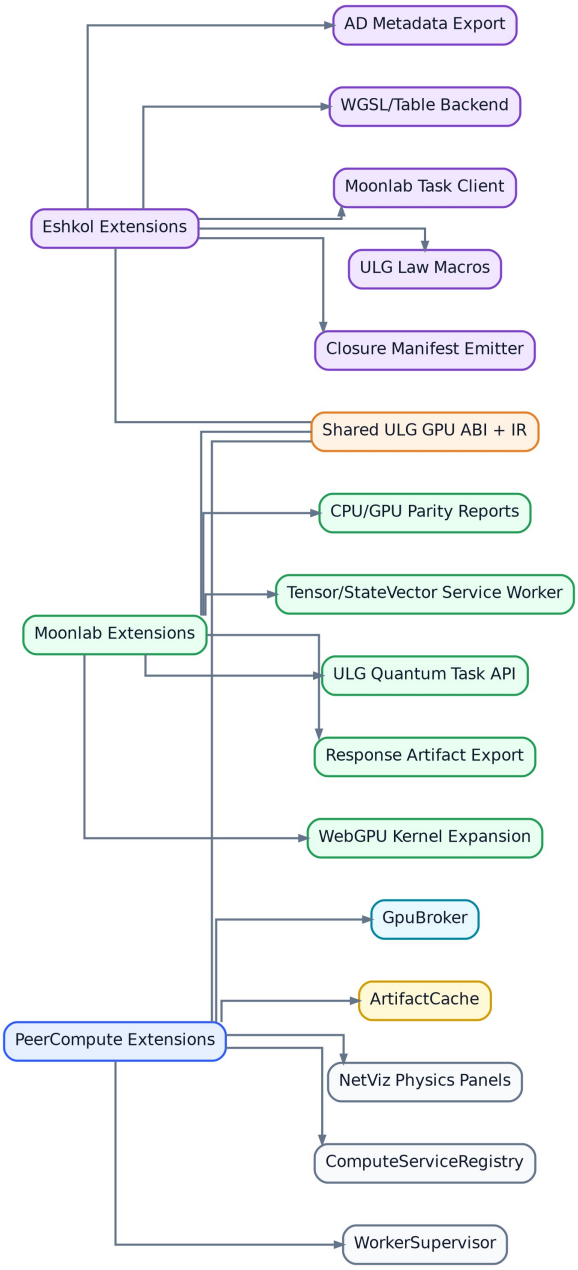


Figure 16: Library extension map

5.3 2. PeerCompute extension plan

5.3.1 2.1 New packages/modules

Recommended modules:

```
peercompute/compute-services/
  ComputeServiceRegistry.ts
  ComputeServiceManifest.ts
  ServiceWorkerHost.ts

peercompute/worker-supervisor/
  WorkerSupervisor.ts
```

```
  ChildWorkerLeaseManager.ts
  WorkerTreeTelemetry.ts
  CancellationTree.ts

peercompute/gpu-broker/
  GpuBroker.ts
  GpuLease.ts
  GpuCapabilityProbe.ts
  DeviceLostRecovery.ts

peercompute/artifacts/
  ArtifactCache.ts
  ClosureRegistry.ts
  ProvenanceIndex.ts
  ValidationStore.ts

peercompute/netviz-panels/
  WorkerTreePanel.tsx
  GpuPressurePanel.tsx
  ClosureCachePanel.tsx
  ProvenancePanel.tsx
  PhysicsResidualPanel.tsx
```

5.3.2 2.2 `ComputeServiceRegistry`

Add service registration for long-lived compute services.

```
await node.compute.registerService({
  serviceId: "eshkol",
  runtime: "wasm-webgpu",
  entry: { workerModule: "/workers/eshkol-service.worker.js" },
  childWorkers: { allowed: true, maxChildren: 8, allowedModules:
 ↪  ["/workers/eshkol-child.worker.js"], sameOriginOnly: true },
  capabilities: ["ulg.closure.derive", "ulg.ad", "ulg.table.generate"],
  abi: { ulgIrVersion: "0.5", gpuAbiVersion: "0.5", supportedDTypes: ["f32",
 ↪  "complex64"], supportedLayouts: ["soa", "interleaved"] },
  validation: { requiresCpuReference: true, toleranceProfile: "scientific-default",
 ↪  parityModes: ["wasm-reference"] }
})
```

5.3.3 2.3 Worker supervision

Add:

- root worker lifecycle;
- child lease request handling;
- heartbeat monitoring;
- cancellation tree;
- worker crash recovery;
- worker tree telemetry for NetViz.

5.3.4 2.4 GPU broker

Add:

- adapter/capability probing;
- lease priority;
- memory pressure estimates;

- render-vs-compute scheduling policy;
- device-lost handling;
- timing hooks;
- WebGPU unsupported fallback.

5.3.5 2.5 Artifact and closure cache

Add:

- content-addressed storage;
- IndexedDB backend;
- closure validity queries;
- invalidation events;
- quarantine status;
- peer cache announcements;
- provenance lookup.

5.3.6 2.6 NetViz extensions

NetViz should visualize:

- active services;
- worker tree;
- child-worker leases;
- GPU leases;
- task lineage;
- artifact provenance;
- closure cache hits/misses;
- validation failures;
- physics residuals.

5.4 3. Eshkol extension plan

5.4.1 3.1 New capabilities

Eshkol should add:

```
ULG law macros
ULG closure manifest emitter
unit/dimensional metadata export
validity-envelope declarations
provenance block emission
Moonlab task client
WASM reference export
WGSL kernel/table strategy backend
closure-table tiling
PeerCompute service worker adapter
```

5.4.2 3.2 Example Eshkol ULG closure shape

```
(define-ulg-closure eos-hydrogen-plasma
  (inputs
    (rho density)
    (T temperature)
    (species species-vector))
  (outputs
    (P pressure)
    (S entropy)
```

```
    (Cv heat-capacity)
    (ion ionization-fraction))
  (assumptions
    local-thermodynamic-equilibrium
    hydrogen-helium-plasma)
  (validity
    (temperature 1e3 1e8)
    (density 1e-12 1e6))
  (body
    (let ((F (+ (ideal-ion-free-energy rho T species)
                (fermi-electron-free-energy rho T species)
                (radiation-free-energy T))))
      (values
        (- (d/d-volume F))
        (- (d/d-temperature F))
        (heat-capacity-from F T)
        (ionization-equilibrium rho T species)))))
```

5.4.3 3.3 Eshkol outputs

For every closure, Eshkol should emit:

1. `ClosureArtifact` manifest;
2. WASM reference function artifact;
3. derivative metadata;
4. optional WGSL module or table-interpolation strategy;
5. validity envelope;
6. uncertainty and validation hints;
7. provenance block;
8. optional Moonlab dependency list.

5.4.4 3.4 Eshkol service tasks

```
eshkol.law.compile
eshkol.closure.derive
eshkol.closure.table.generate
eshkol.derivative.export
eshkol.wasm.reference.build
eshkol.wgsl.emit
eshkol.validation.reference
```

5.4.5 3.5 Eshkol acceptance tests

- Compile a ULG closure into a manifest.
- Export a WASM reference function.
- Generate a tiled closure table with child workers.
- Validate random table samples against WASM reference.
- Emit derivative metadata.
- Request a Moonlab quantum-response task through PeerCompute.

5.5 4. Moonlab extension plan

5.5.1 4.1 New capabilities

Moonlab should add:

```
ULG quantum task API
ULG response artifact exporter
service worker bootstrap
WebGPU capability report in service manifest
CPU/WebGPU parity report
complex64 tensor ABI support
sparse Hamiltonian matvec task
Lanczos/Krylov task
state-vector and tensor-network task adapters
transition/correlation response tasks
```

5.5.2 4.2 Moonlab service tasks

```
moonlab.quantum.response
moonlab.quantum.spectrum
moonlab.quantum.ground_state
moonlab.quantum.transition_matrix
moonlab.quantum.correlation
moonlab.tensor_network.contract
moonlab.tensor_network.expectation
moonlab.parity.cpu_webgpu
```

5.5.3 4.3 Moonlab artifact shape

```
{
  "artifactId": "quantum.response.local_patch.001",
  "sourceService": "moonlab",
  "taskKind": "quantum.response",
  "method": "lanczos-demo",
  "representation": "state_vector",
  "outputs": {
    "energyLevels": "artifact://...",
    "transitionMatrixElements": "artifact://...",
    "forceSamples": "artifact://..."
  },
  "uncertainty": {
    "truncationError": 0.0,
    "parityError": 0.0
  },
  "validation": {
    "status": "pass",
    "validationMode": "cpu-reference"
  }
}
```

5.5.4 4.4 Moonlab WebGPU priorities

Prioritize kernels that directly help ULG closure generation:

1. complex vector operations;
2. complex dot/norm reductions;
3. sparse Hamiltonian matvec;
4. Krylov/Lanczos loop primitives;
5. tensor contraction primitives;
6. MPS two-site updates;
7. expectation-value reductions;

8. probability reductions;
9. CPU/GPU parity harness.

5.5.5 4.5 Moonlab acceptance tests

- Run a small CPU quantum task.
- Run the same task on WebGPU where available.
- Emit a parity report.
- Emit a `QuantumResponseArtifact`.
- Quarantine failed parity outputs.
- Feed a response artifact to Eshkol closure derivation.

5.6 5. Shared ULG GPU ABI package

Create a shared package used by all three libraries.

```
ulg-gpu-abi/
  package.json
  src/types.ts
  src/layouts/complex64.ts
  src/layouts/tensor.ts
  src/layouts/closure-table.ts
  src/wgsl/common.wgsl
  schemas/*.json
  conformance/*.test.ts
```

5.6.1 Required descriptors

- `DTypeDescriptor`;
- `TensorDescriptor`;
- `ComplexLayoutDescriptor`;
- `ClosureTableDescriptor`;
- `KernelModuleDescriptor`;
- `ToleranceReport`;
- `ProvenanceBlock`.

5.7 6. Cross-repository integration plan

5.7.1 Step 1 — publish ABI package

All repos import the shared types. No GPU backend work proceeds until the ABI package exists.

5.7.2 Step 2 — dummy service integration

PeerCompute registers dummy Eshkol and Moonlab services that return mock artifacts.

5.7.3 Step 3 — real service bootstraps

Eshkol and Moonlab run as WASM service workers under PeerCompute.

5.7.4 Step 4 — real child-worker leases

Both services use leased child workers rather than unsupervised spawning.

5.7.5 Step 5 — real artifacts

Moonlab emits a quantum artifact. Eshkol emits a closure artifact. PeerCompute caches and displays both.

5.7.6 Step 6 — ULG consumes closure

A toy ULG WebGPU kernel uses the closure table and emits compact deltas.

5.8 7. Backward compatibility

- PeerCompute keeps existing compute-task APIs; service registry is additive.
- Eshkol keeps existing language/compiler behavior; ULG macros are additive.
- Moonlab keeps existing bindings; ULG task API is additive.
- WebGPU support remains optional with CPU/WASM fallback.

5.9 8. Repository ownership boundaries

Concern	Owner
Network authority	PeerCompute
Distributed scheduling	PeerCompute
Worker tree	PeerCompute
GPU leases	PeerCompute
Law language	Eshkol
AD / closure math	Eshkol
Closure artifact emission	Eshkol
Quantum state/tensor simulation	Moonlab
Quantum response artifact emission	Moonlab
Shared tensor/complex ABI	shared package
Hot carrier kernels	ULG runtime / PeerCompute service
Visualization	PeerCompute / NetViz

5.10 9. What not to build

Do not build:

- an independent Eshkol P2P scheduler;
- an independent Moonlab P2P scheduler;
- separate GPU ABIs for Eshkol and Moonlab;
- hidden child-worker spawns outside PeerCompute;
- full tensor replication through network messages;
- artifact caches without provenance;
- WebGPU-only correctness paths without CPU/WASM fallback.

5.11 Source notes

This specification is based on the public project documentation checked on 2026-06-05:

- PeerCompute repository and README
- Eshkol repository and README
- Moonlab repository and README
- Moonlab architecture notes
- Moonlab WebGPU plan
- Web Workers documentation
- Worker WebGPU entry point
- SharedArrayBuffer isolation requirements
- WGSL specification

---

6 Appendix A — Standalone Mermaid Diagram Sources

These are the standalone Mermaid diagram sources from the v0.5 package, embedded so the file can be rendered without external `.mmd` files.

6.1 01_system_context.mmd

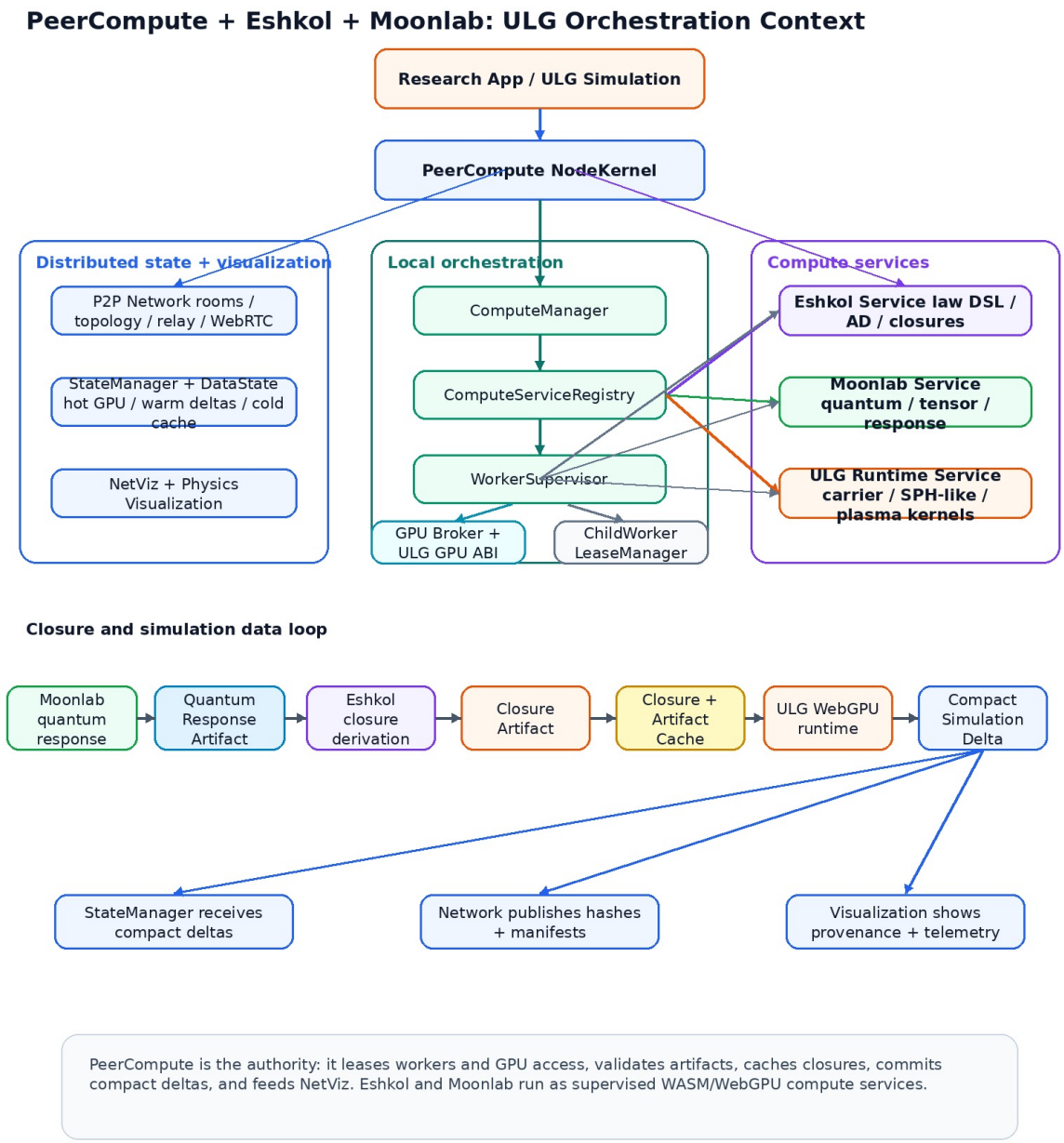


Figure 17: System context and orchestration loop

6.2 02_supervised_worker_tree.mmd

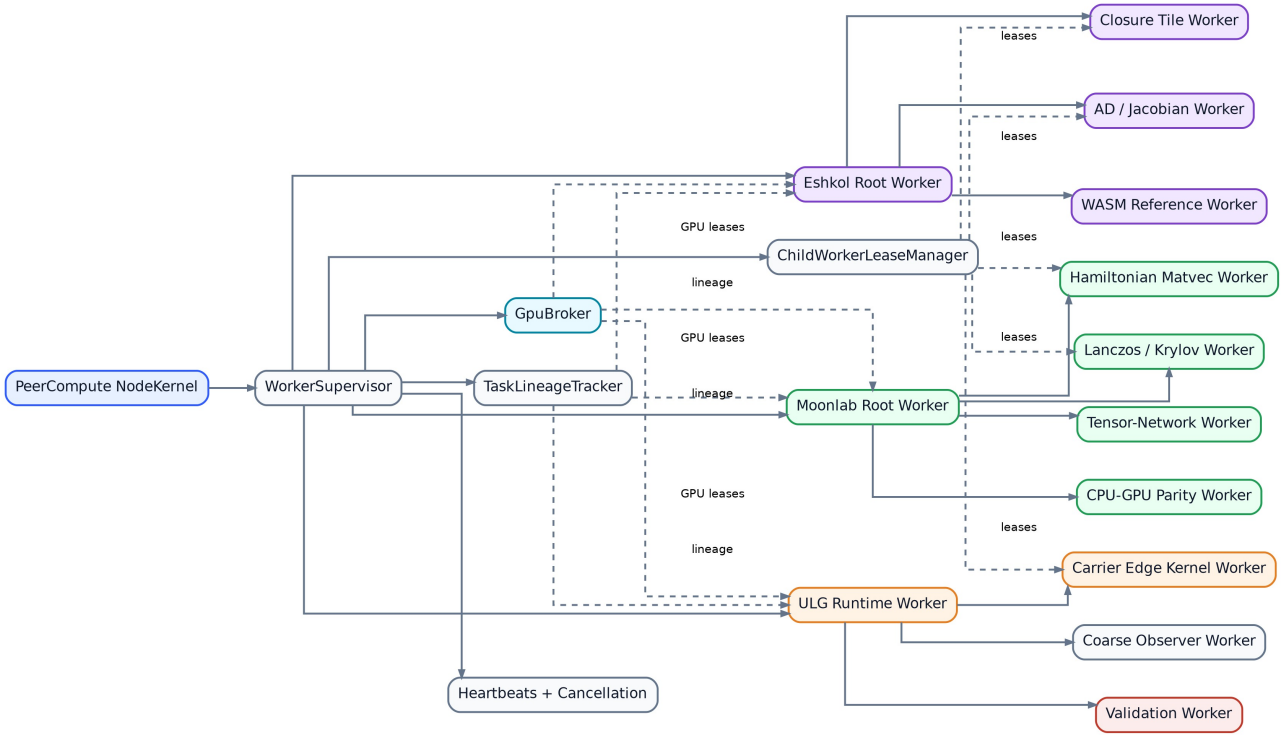


Figure 18: Supervised worker tree

6.3 03_closure_factory_flow.mmd

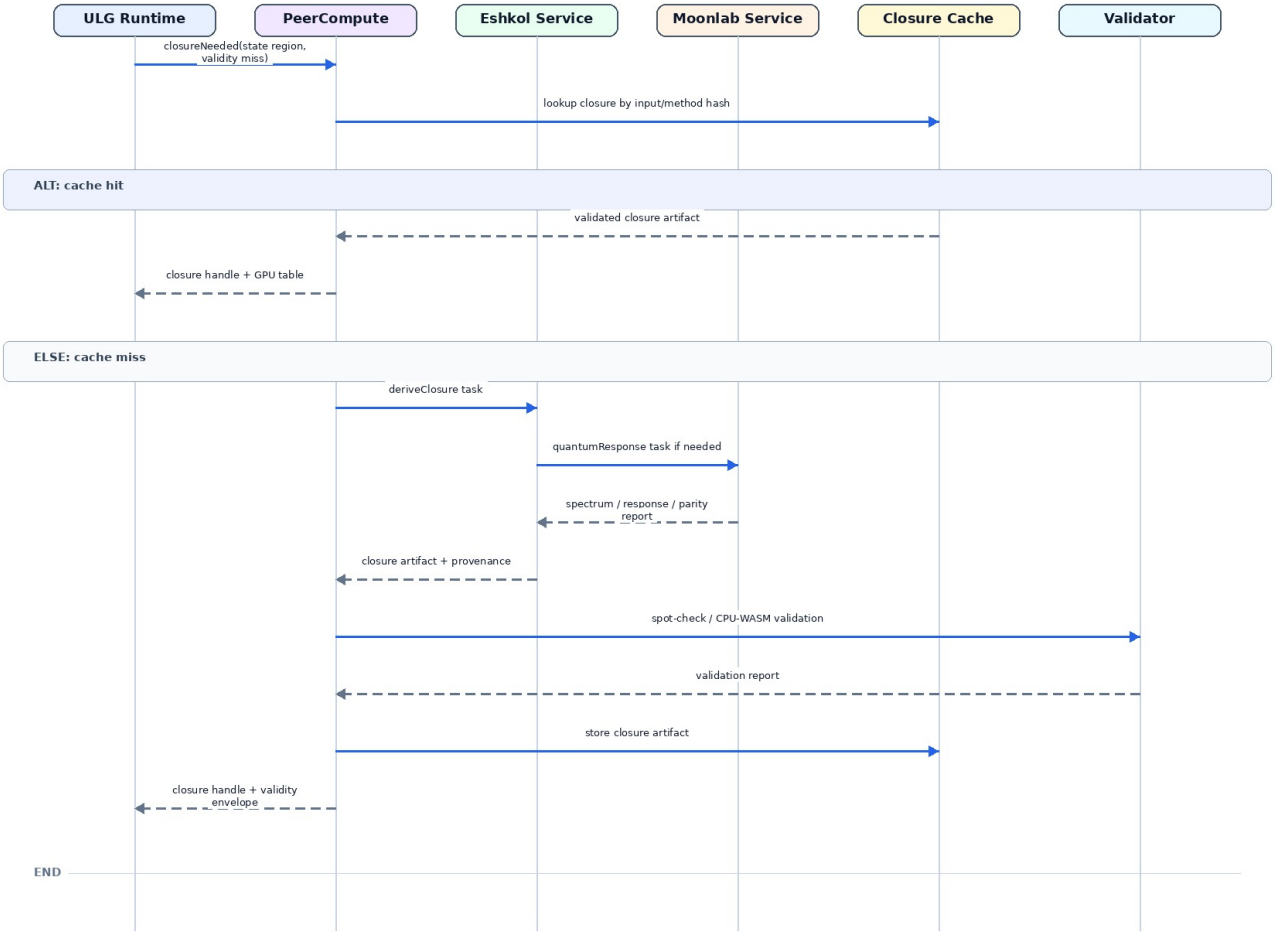


Figure 19: Closure derivation sequence

6.4 04_shared_gpu_abi.mmd

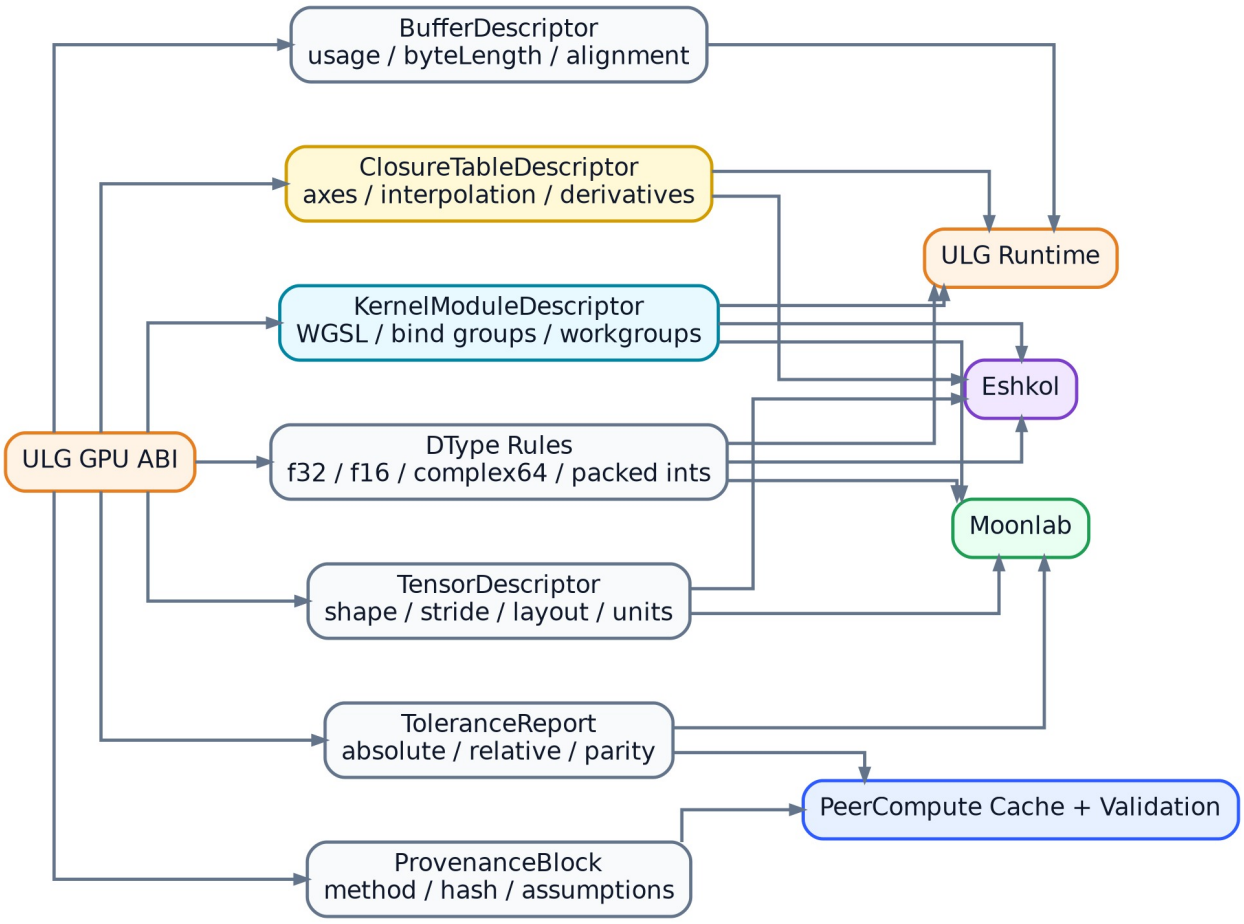


Figure 20: Shared ULG GPU ABI

6.5 05_data_lifecycle.mmd

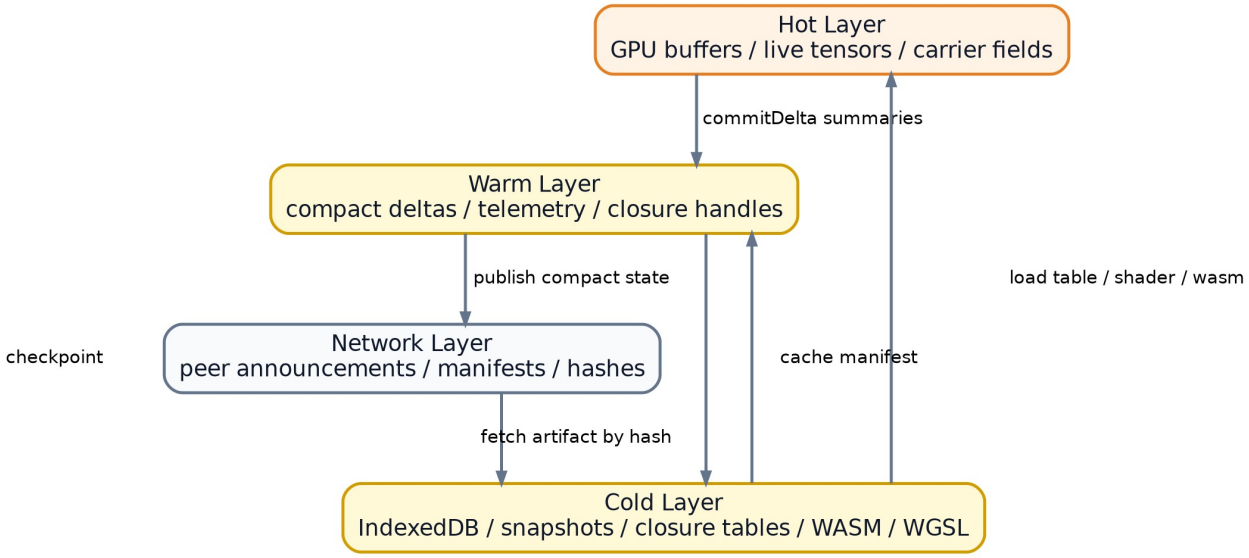


Figure 21: Data lifecycle across hot, warm, cold, and network layers

6.6 06_task_state_machine.mmd

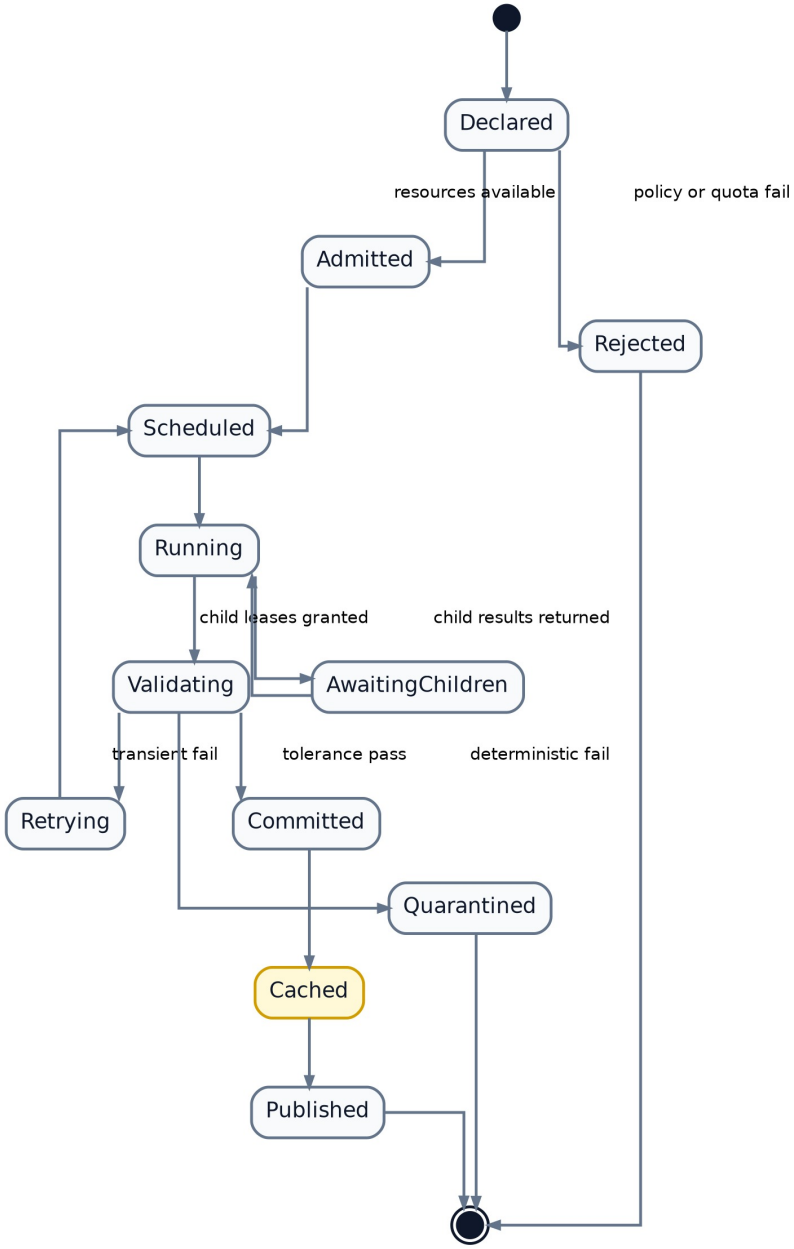


Figure 22: Task lifecycle state machine

6.7 07_validation_provenance_loop.mmd

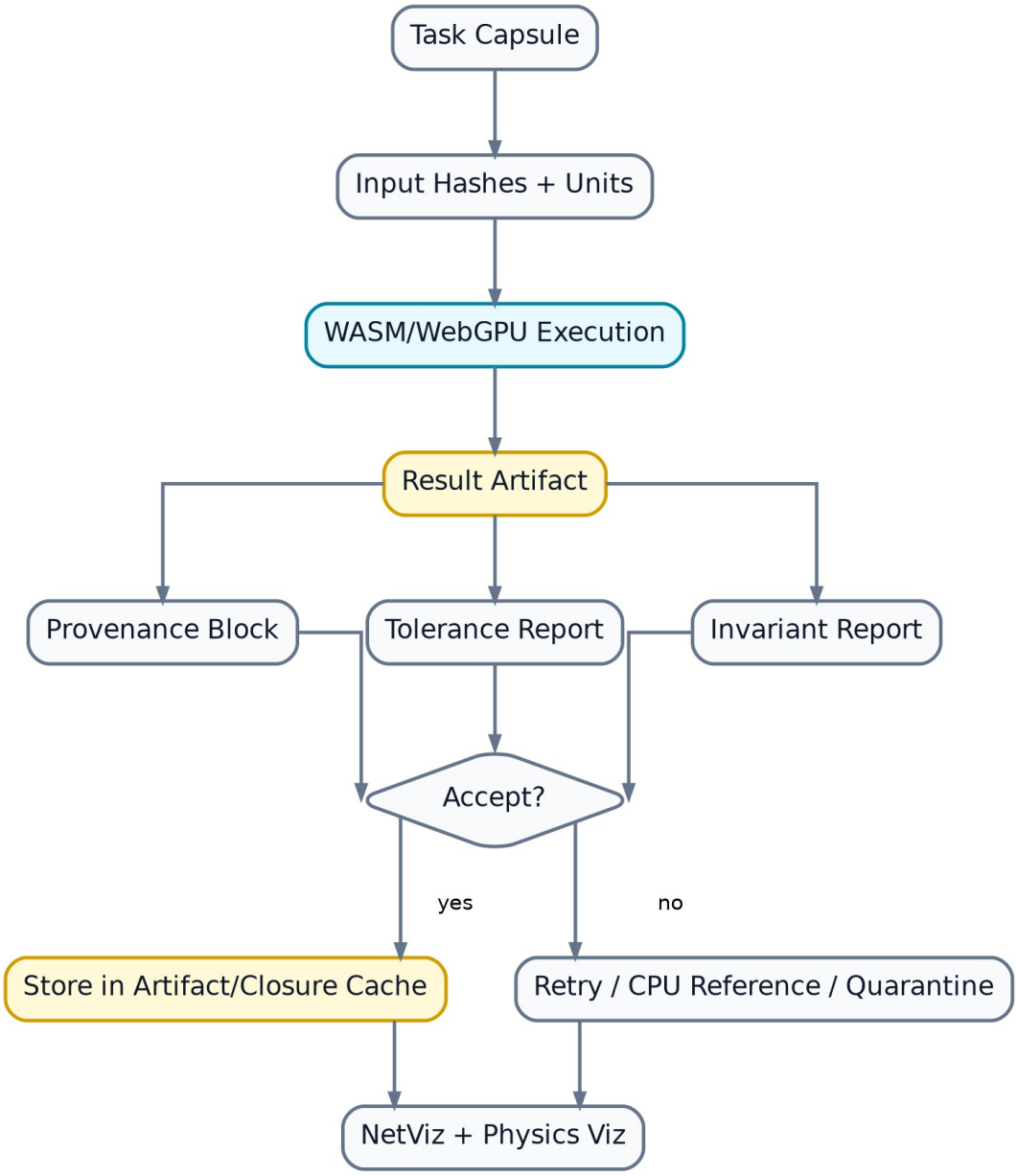


Figure 23: Validation and provenance loop

6.8 08_library_extension_map.mmd

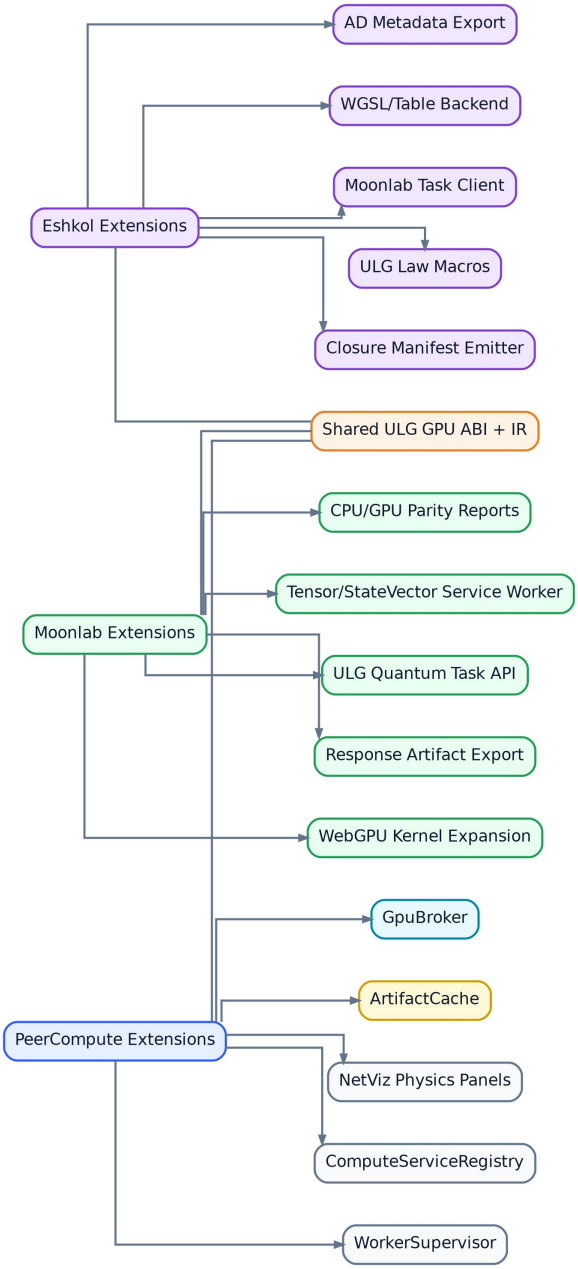


Figure 24: Library extension map

6.9 09_roadmap.mmd

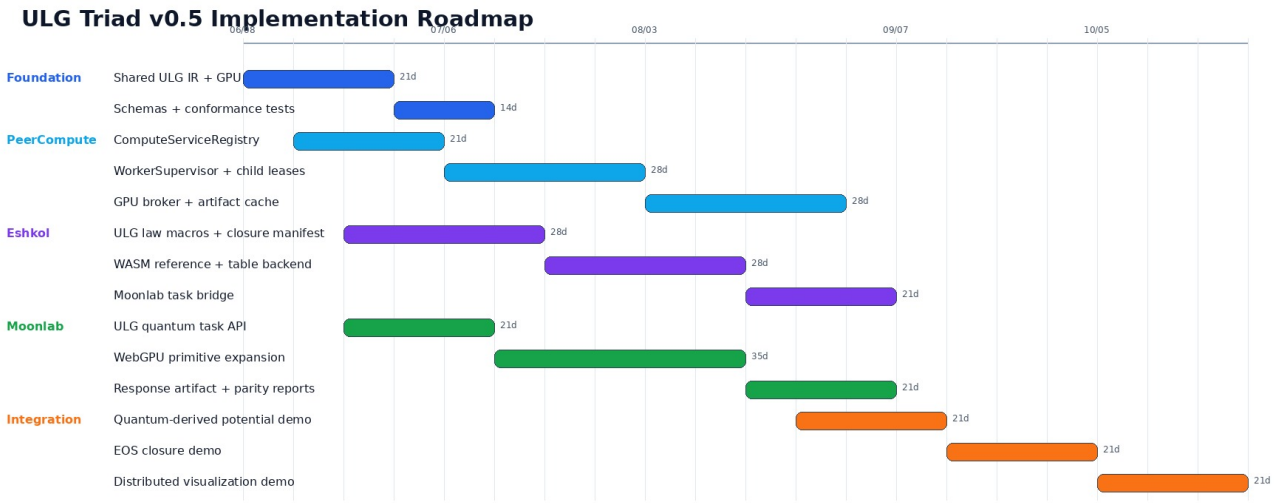


Figure 25: ULG Triad v0.5 Implementation Roadmap

6.10 10_visualization_telemetry.mmd

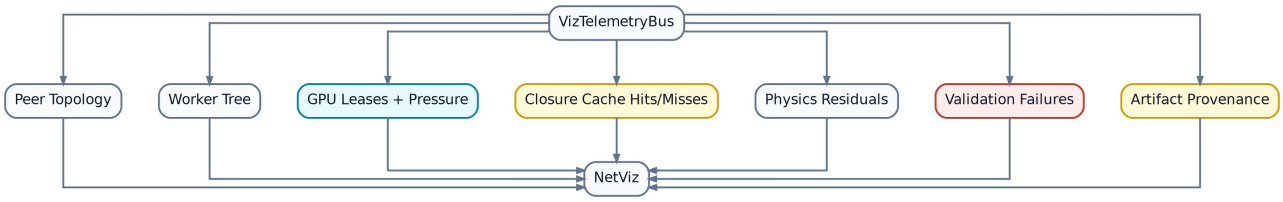


Figure 26: Visualization telemetry bus

6.11 11_security_resource_model.mmd

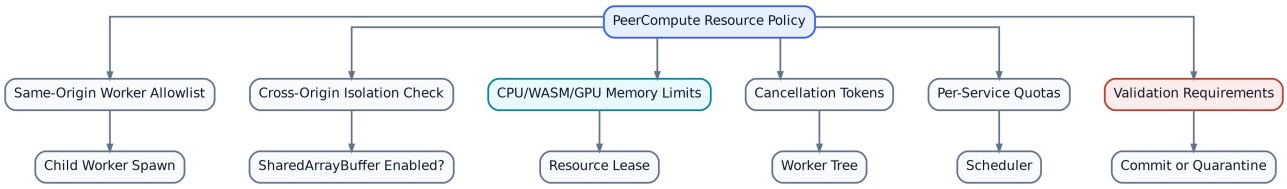


Figure 27: Resource and security policy

6.12 12_end_to_end_demo.mmd

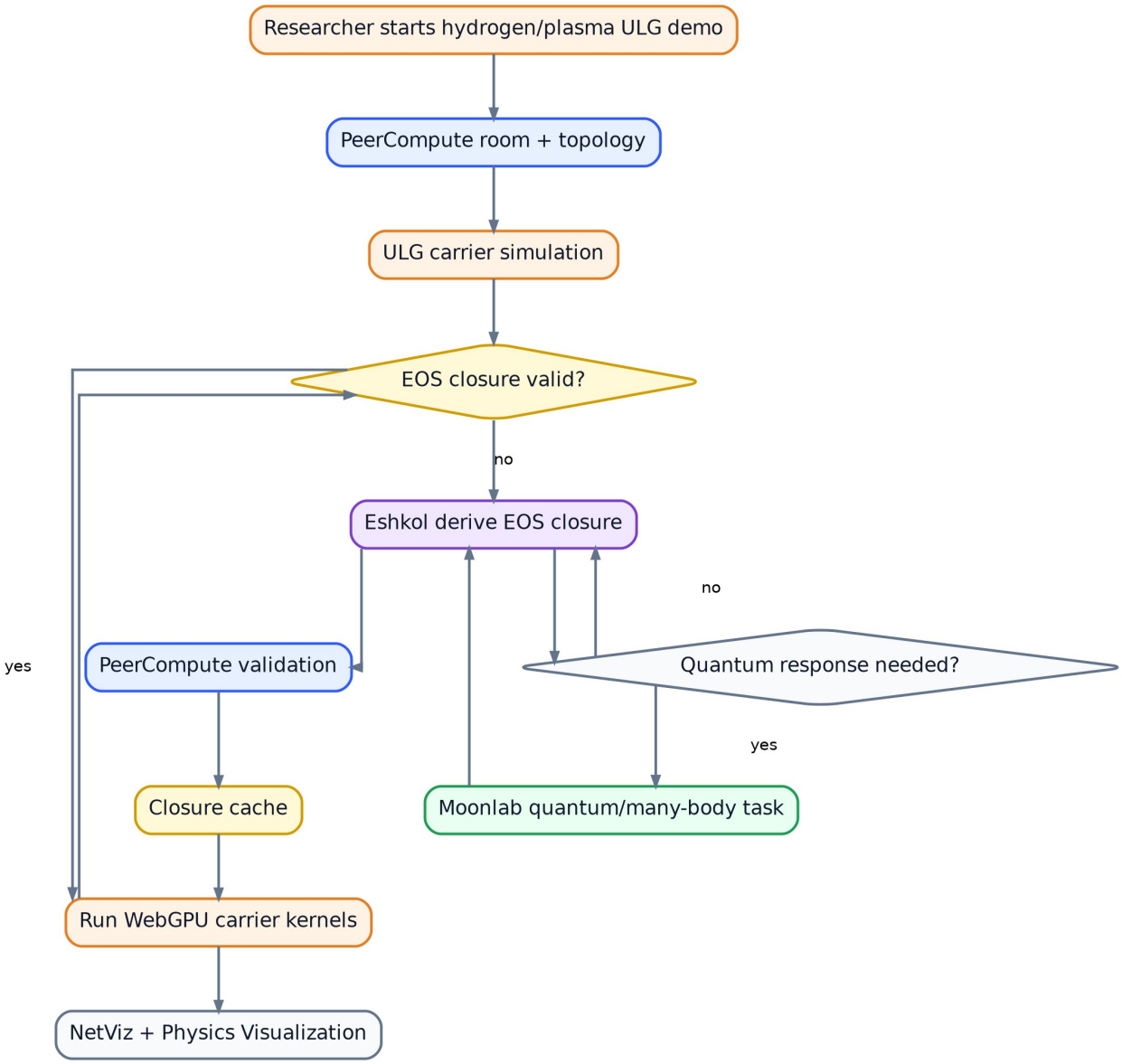


Figure 28: Hydrogen plasma demo flow

6.13 Diagram index notes

Version: **0.5**
Date: **2026-06-05**

This directory contains standalone Mermaid source diagrams used by the v0.5 Markdown documents.

6.14 01 System Context

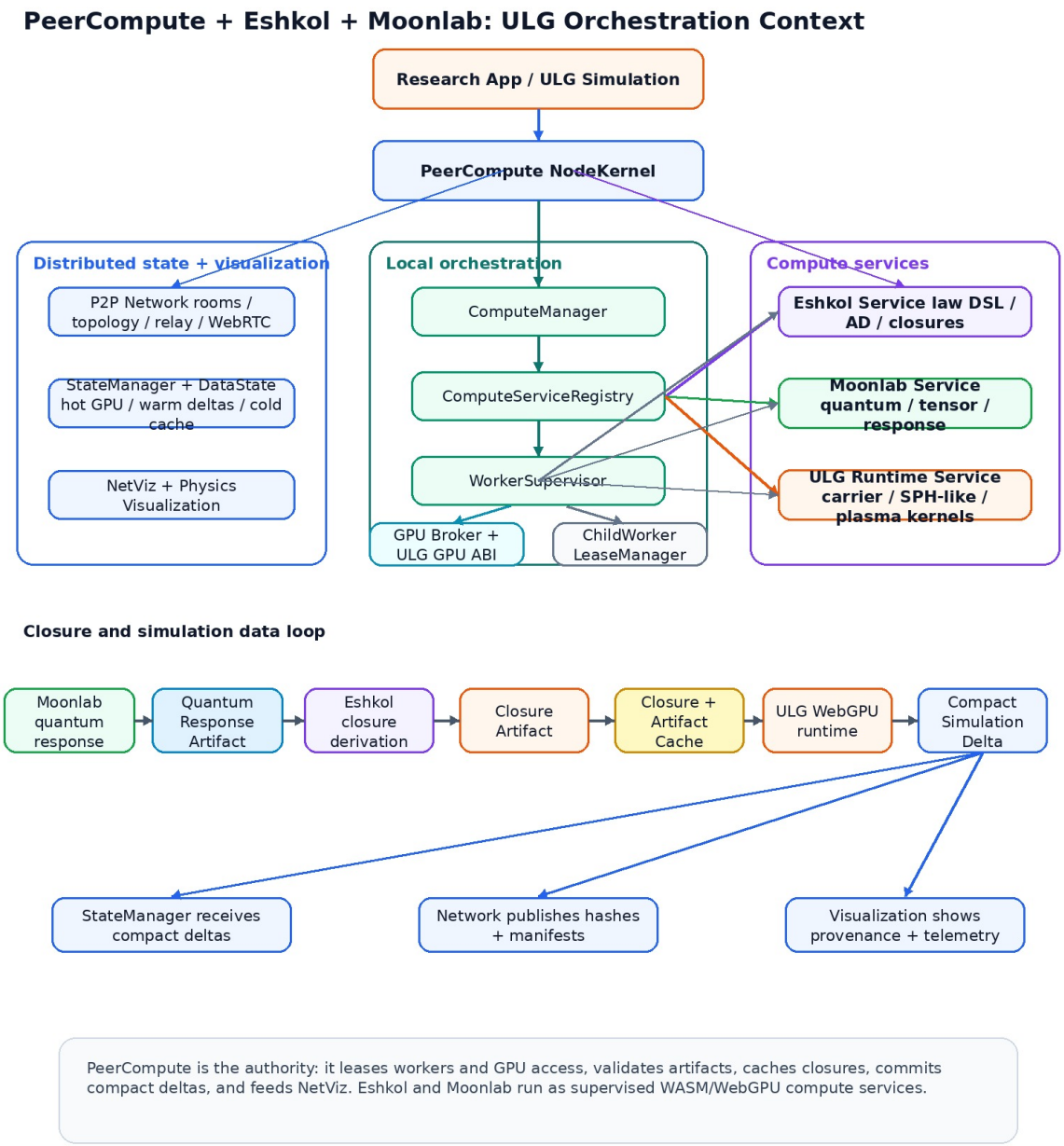


Figure 29: System context and orchestration loop

File: `01_system_context.mmd`

6.15 02 Supervised Worker Tree

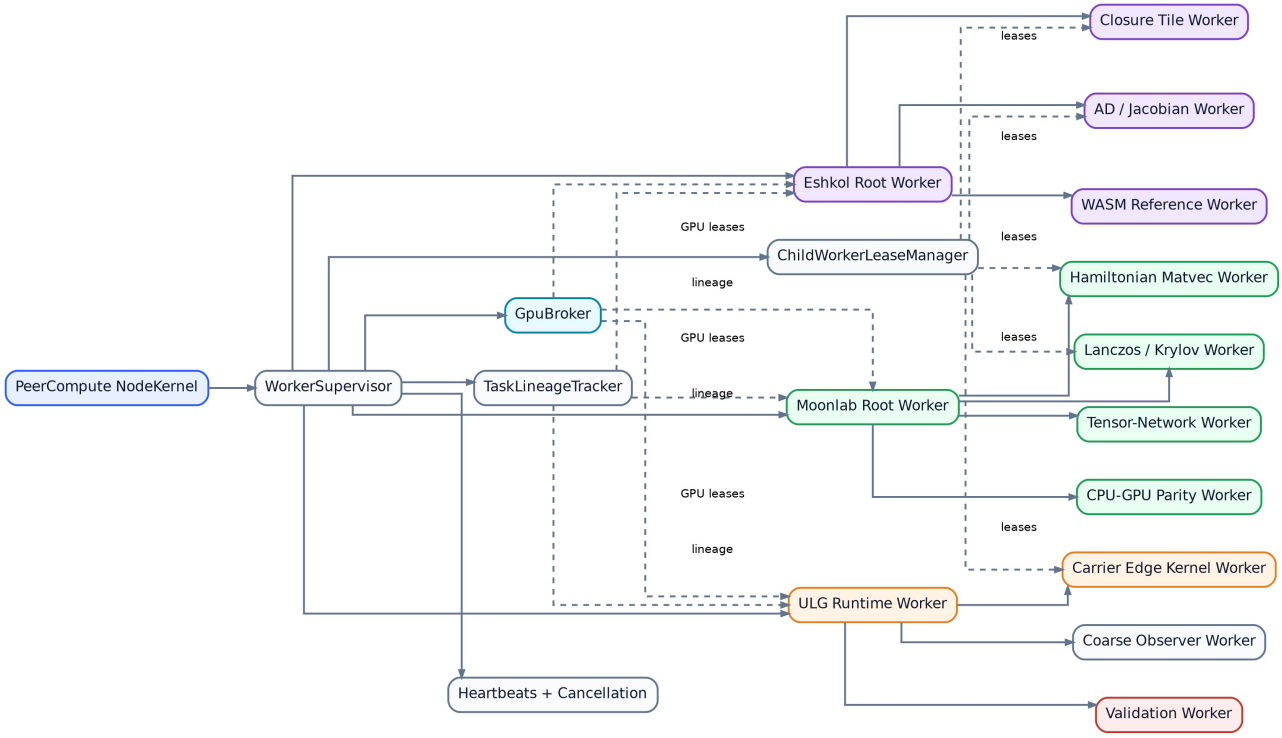


Figure 30: Supervised worker tree

File: `02_supervised_worker_tree.mmd`

6.16 03 Closure Factory Flow

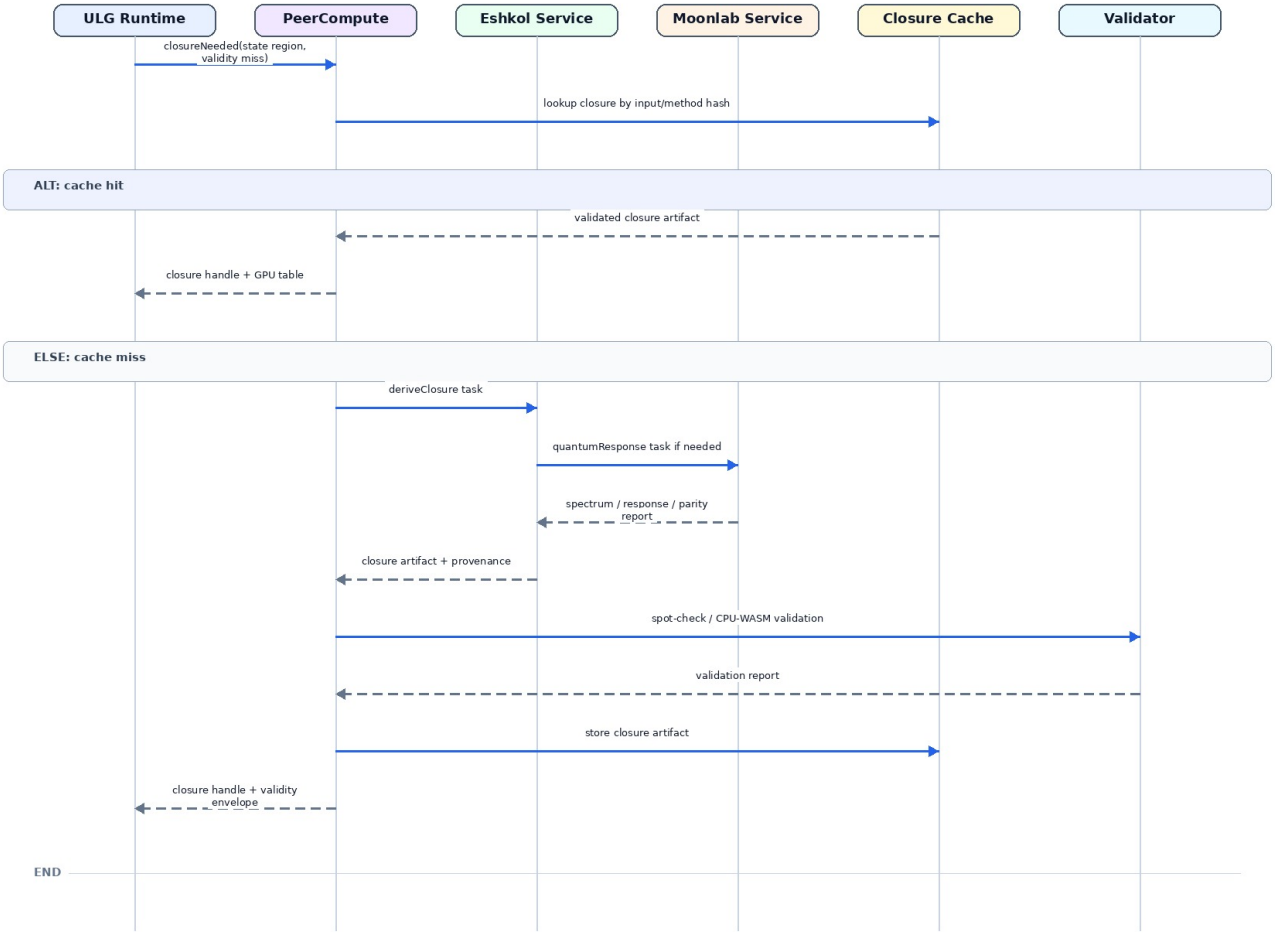


Figure 31: Closure derivation sequence

File: `03_closure_factory_flow.mmd`

6.17 04 Shared Gpu Abi

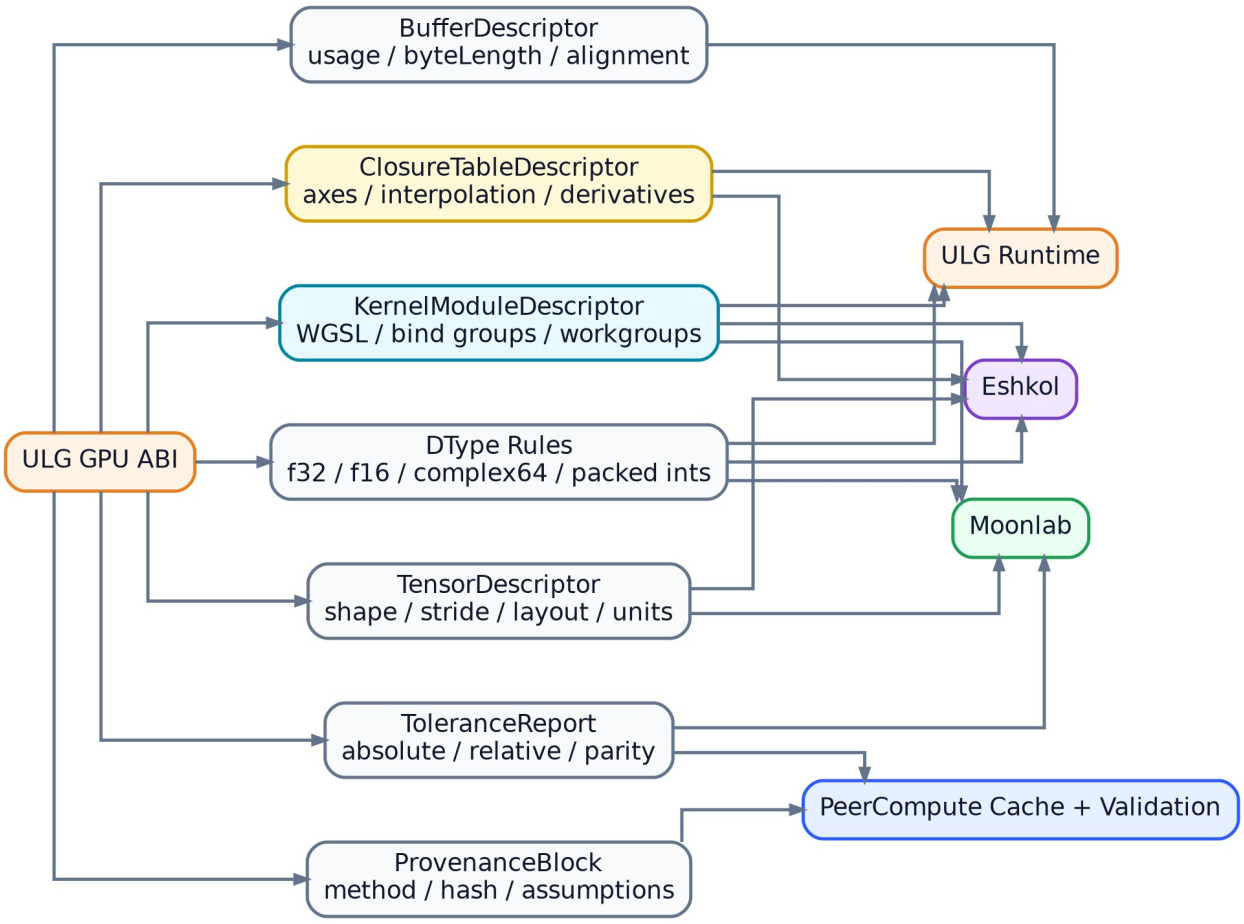


Figure 32: Shared ULG GPU ABI

File: `04_shared_gpu_abi.mmd`

6.18 05 Data Lifecycle

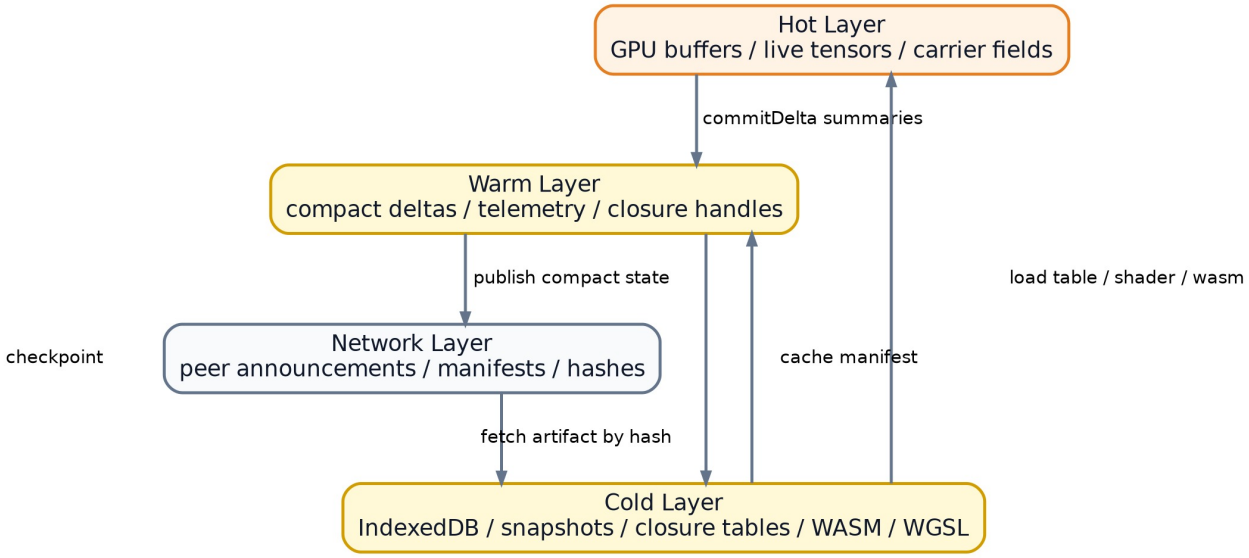


Figure 33: Data lifecycle across hot, warm, cold, and network layers

File: `05_data_lifecycle.mmd`

6.19 06 Task State Machine

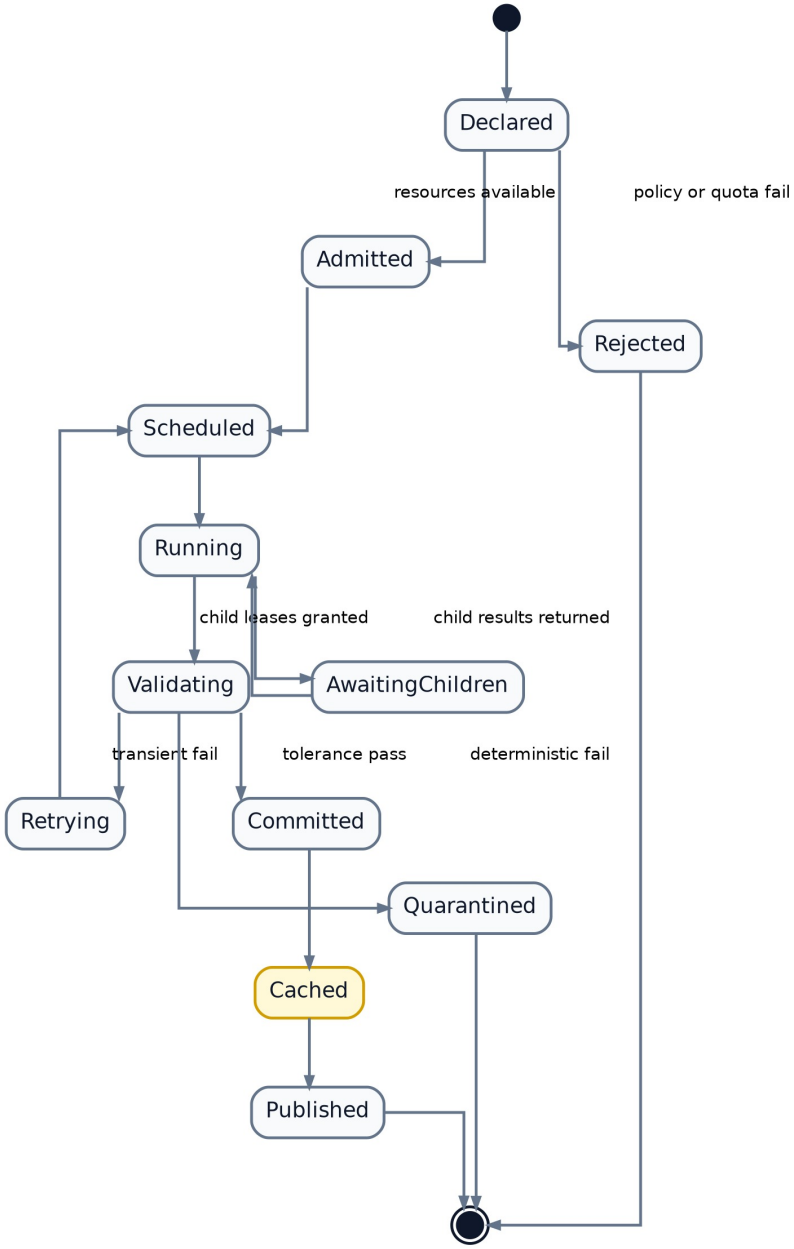


Figure 34: Task lifecycle state machine

File: 06_task_state_machine.mmd

6.20 07 Validation Provenance Loop

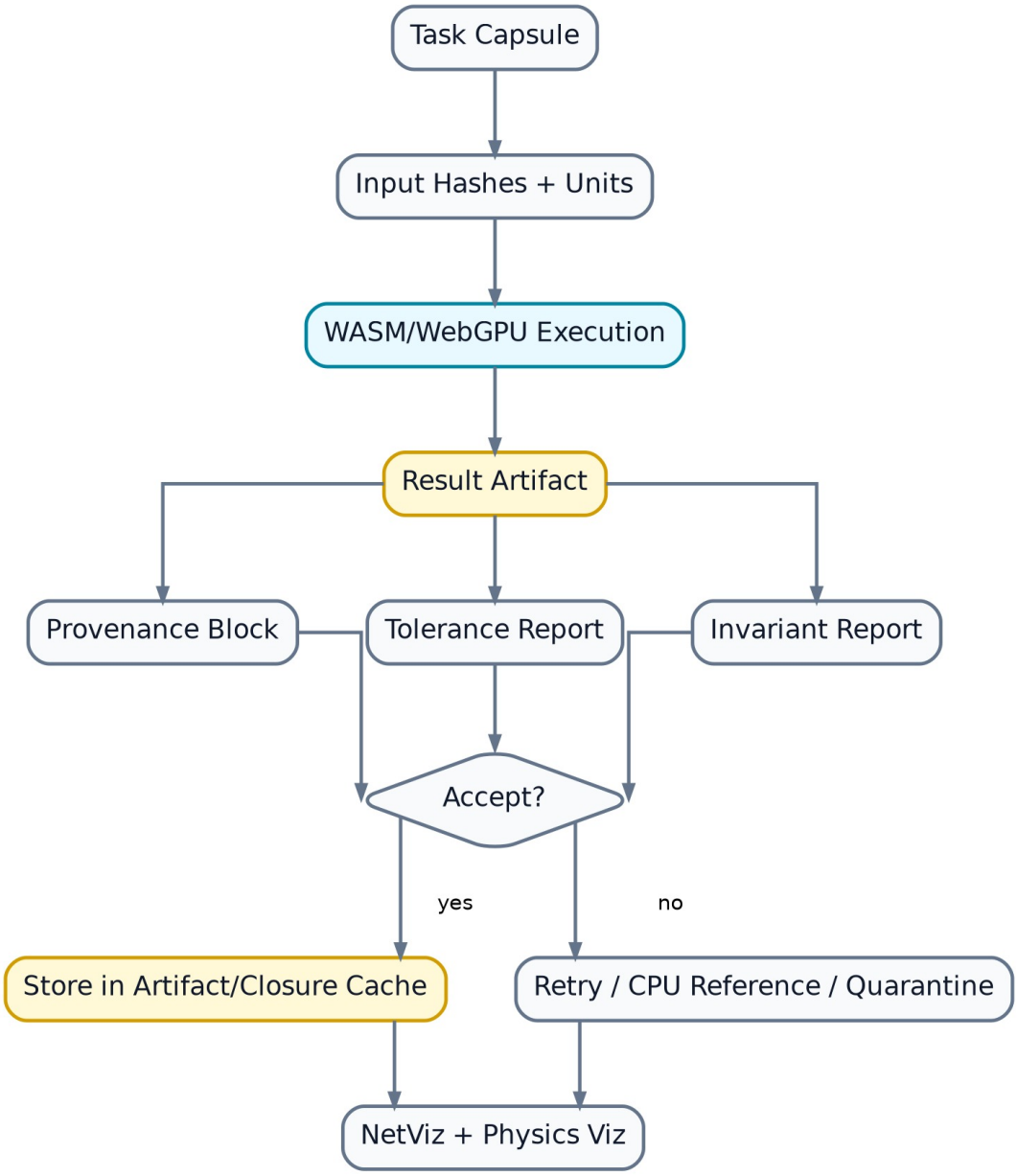


Figure 35: Validation and provenance loop

File: 07_validation_provenance_loop.mmd

6.21 08 Library Extension Map

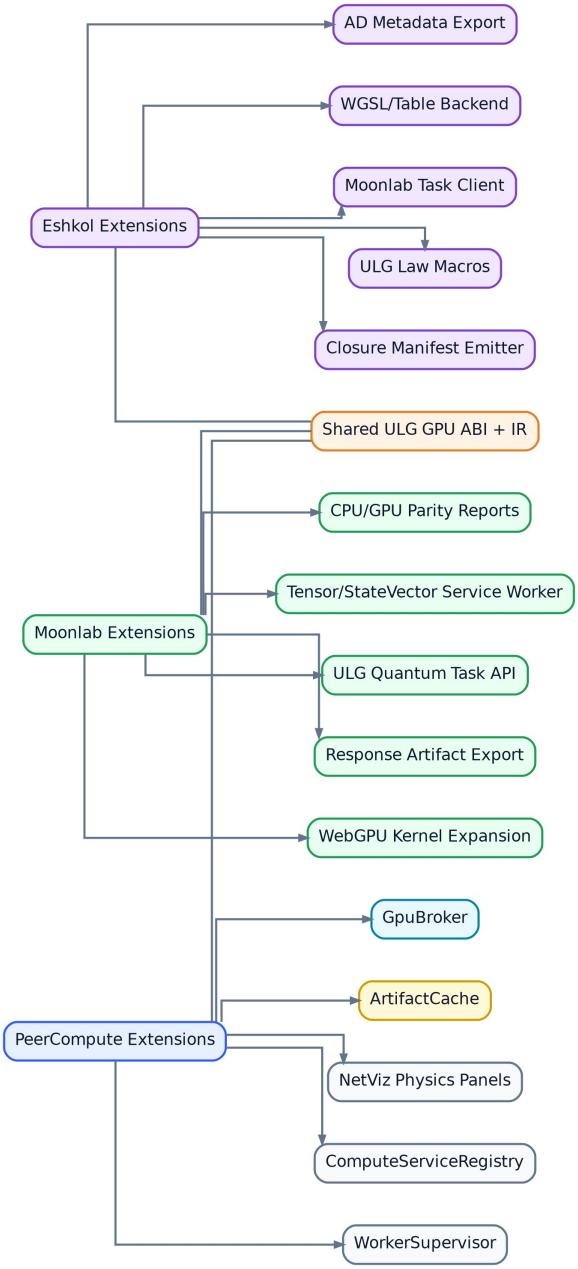


Figure 36: Library extension map

File: `08_library_extension_map.mmd`

6.22 09 Roadmap

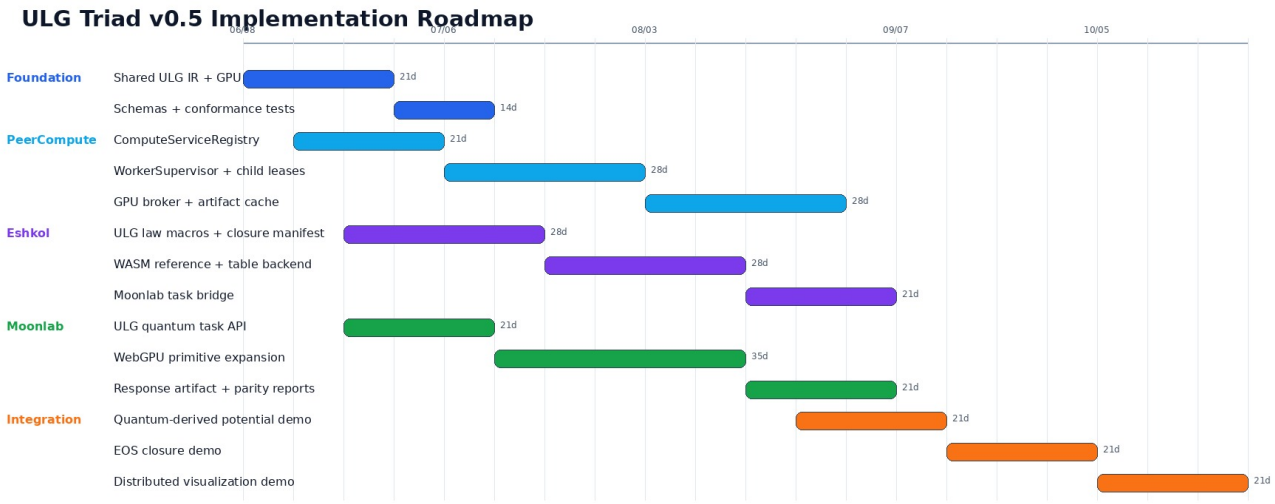


Figure 37: ULG Triad v0.5 Implementation Roadmap

File: `09_roadmap.mmd`

6.23 10 Visualization Telemetry

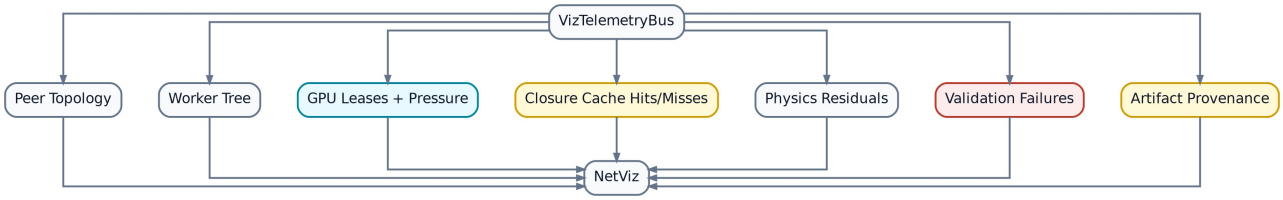


Figure 38: Visualization telemetry bus

File: `10_visualization_telemetry.mmd`

6.24 11 Security Resource Model

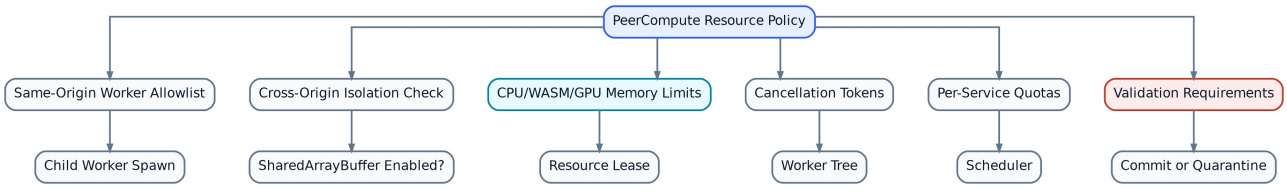


Figure 39: Resource and security policy

File: `11_security_resource_model.mmd`

6.25 12 End To End Demo

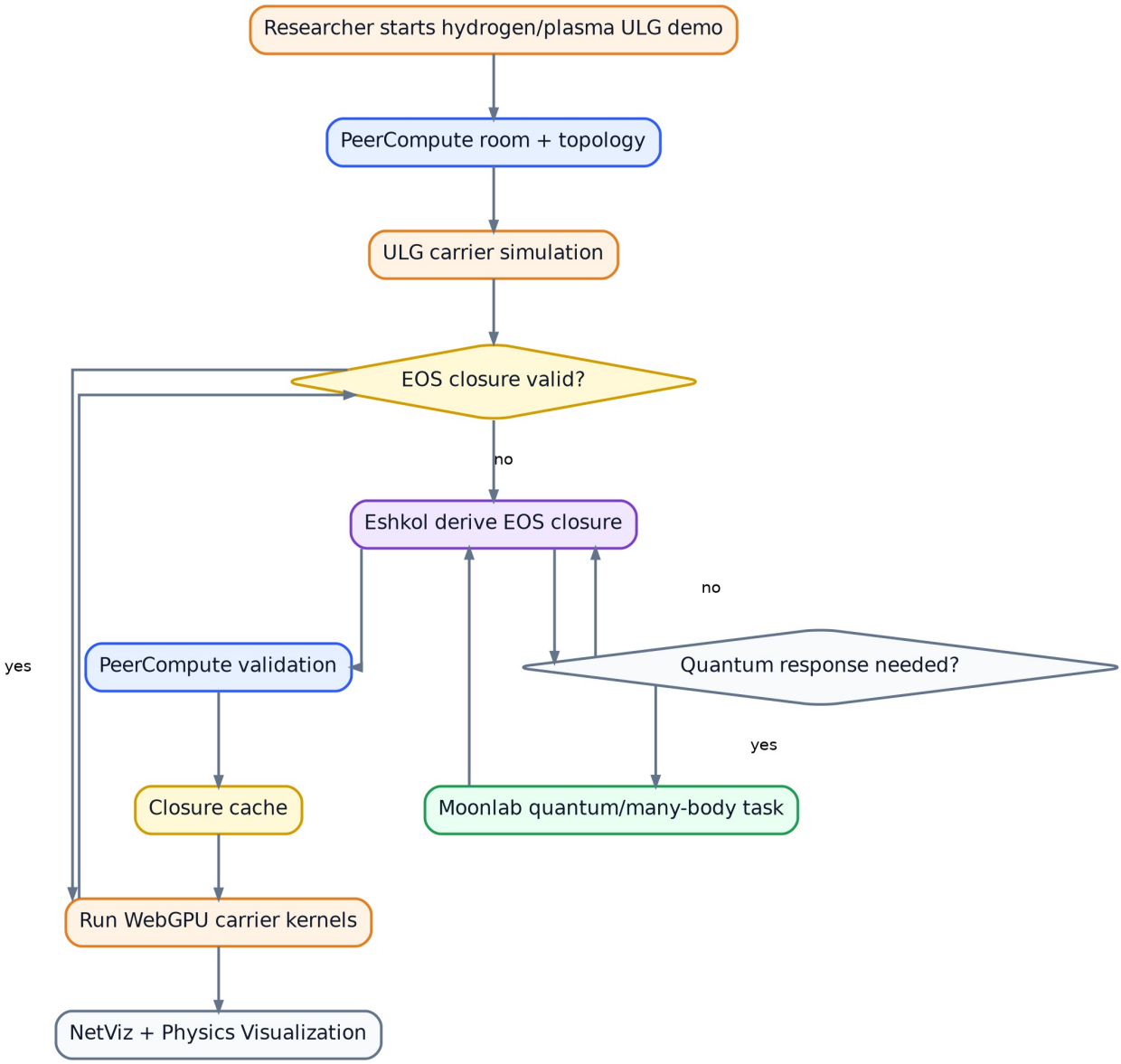


Figure 40: Hydrogen plasma demo flow

File: `12_end_to_end_demo.mmd`

---

7 Appendix B — JSON Schema Sketches

These schema sketches define the initial interchange contracts for compute services, task capsules, closure artifacts, quantum-response artifacts, and validation reports.

7.1 Schema index notes

These schemas are intentionally initial sketches. They exist to make the v0.5 contracts concrete enough to test across PeerCompute, Eshkol, and Moonlab.

- closure_artifact.schema.json
- compute_service_manifest.schema.json
- quantum_response_artifact.schema.json
- task_capsule.schema.json
- validation_report.schema.json

7.2 closure_artifact.schema.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "ULG Closure Artifact v0.5",
  "type": "object",
  "required": [
    "closureId",
    "sourceService",
    "closureKind",
    "inputs",
    "outputs",
    "execution",
    "validity",
    "provenance"
  ],
  "properties": {
    "closureId": {
      "type": "string"
    },
    "sourceService": {
      "type": "string"
    },
    "closureKind": {
      "type": "string"
    },
    "inputs": {
      "type": "array"
    },
    "outputs": {
      "type": "array"
    },
    "derivatives": {
      "type": "array"
    },
    "execution": {
      "type": "object"
    },
    "validity": {
      "type": "object"
    },
    "uncertainty": {
      "type": "object"
    },
    "validation": {
      "type": "object"
    },
```

```
    "provenance": {
      "type": "object"
    }
  }
}
```

7.3 compute_service_manifest.schema.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "ULG Compute Service Manifest v0.5",
  "type": "object",
  "required": [
    "serviceId",
    "version",
    "runtime",
    "entry",
    "capabilities",
    "abi"
  ],
  "properties": {
    "serviceId": {
      "type": "string"
    },
    "version": {
      "type": "string"
    },
    "runtime": {
      "enum": [
        "wasm",
        "webgpu",
        "wasm-webgpu",
        "js",
        "native-bridge"
      ]
    },
    "entry": {
      "type": "object",
      "required": [
        "workerModule"
      ],
      "properties": {
        "workerModule": {
          "type": "string"
        },
        "wasmModule": {
          "type": "string"
        },
        "initExport": {
          "type": "string"
        }
      }
    },
    "childWorkers": {
      "type": "object"
    },
```

```
    "resources": {
      "type": "object"
    },
    "capabilities": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "abi": {
      "type": "object"
    },
    "validation": {
      "type": "object"
    }
  }
}
```

7.4 quantum_response_artifact.schema.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "ULG Quantum Response Artifact v0.5",
  "type": "object",
  "required": [
    "artifactId",
    "sourceService",
    "taskKind",
    "inputHash",
    "method",
    "representation",
    "outputs",
    "validation",
    "provenance"
  ],
  "properties": {
    "artifactId": {
      "type": "string"
    },
    "sourceService": {
      "const": "moonlab"
    },
    "taskKind": {
      "type": "string"
    },
    "inputHash": {
      "type": "string"
    },
    "method": {
      "type": "string"
    },
    "representation": {
      "type": "string"
    },
    "outputs": {
      "type": "object"
```

```
    },
    "uncertainty": {
      "type": "object"
    },
    "validation": {
      "type": "object"
    },
    "provenance": {
      "type": "object"
    }
  }
}
```

7.5 task_capsule.schema.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "ULG Task Capsule v0.5",
  "type": "object",
  "required": [
    "taskId",
    "rootTaskId",
    "serviceId",
    "taskKind",
    "inputHash",
    "methodHash"
  ],
  "properties": {
    "taskId": {
      "type": "string"
    },
    "rootTaskId": {
      "type": "string"
    },
    "serviceId": {
      "type": "string"
    },
    "taskKind": {
      "type": "string"
    },
    "inputs": {
      "type": "array"
    },
    "outputs": {
      "type": "array"
    },
    "inputHash": {
      "type": "string"
    },
    "methodHash": {
      "type": "string"
    },
    "resources": {
      "type": "object"
    },
    "validation": {
```

```
      "type": "object"
    },
    "provenance": {
      "type": "object"
    }
  }
}
```

7.6 validation_report.schema.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "ULG Validation Report v0.5",
  "type": "object",
  "required": [
    "status",
    "validationMode",
    "toleranceProfile"
  ],
  "properties": {
    "status": {
      "enum": [
        "unchecked",
        "pass",
        "warn",
        "fail",
        "quarantined"
      ]
    },
    "validatorPeer": {
      "type": "string"
    },
    "validationMode": {
      "type": "string"
    },
    "toleranceProfile": {
      "type": "string"
    },
    "maxAbsError": {
      "type": "number"
    },
    "maxRelError": {
      "type": "number"
    },
    "invariantDrift": {
      "type": "object"
    },
    "notes": {
      "type": "array",
      "items": {
        "type": "string"
      }
    }
  }
}
```

From dynamic-law refactor to real-time physical breadth

The worker-owned SS lane, authenticated reaction activation, and truthful native presentation are the foundation. The immediate program now restores real-time cadence, makes steam and aerosol transport physically visible, and then broadens solids, law activation, reservations, and online closures.

Immediate priority: restore at least 1× simulation time across every canned preset without weakening exact physics or presentation authority.

FOUNDATION LANDED

One canonical execution spine

SS spatial authority → worker-resident lane → exact schedule/fence → StateManager commit → versioned native presentation.

GOAL 1 IN PROGRESS

Throughput + real presentation

Measure all six presets honestly, remove serialized presentation stalls, and retain periodic real-desktop animation checks.

GOALS 2–7 QUEUED

Physical breadth + scale

Visible steam/aerosol transport, coherent solids, general per-node laws, distributed reservations, online closures, then an external fuzzy release audit.

6
canned presets in the controlled throughput and desktop-visual matrix

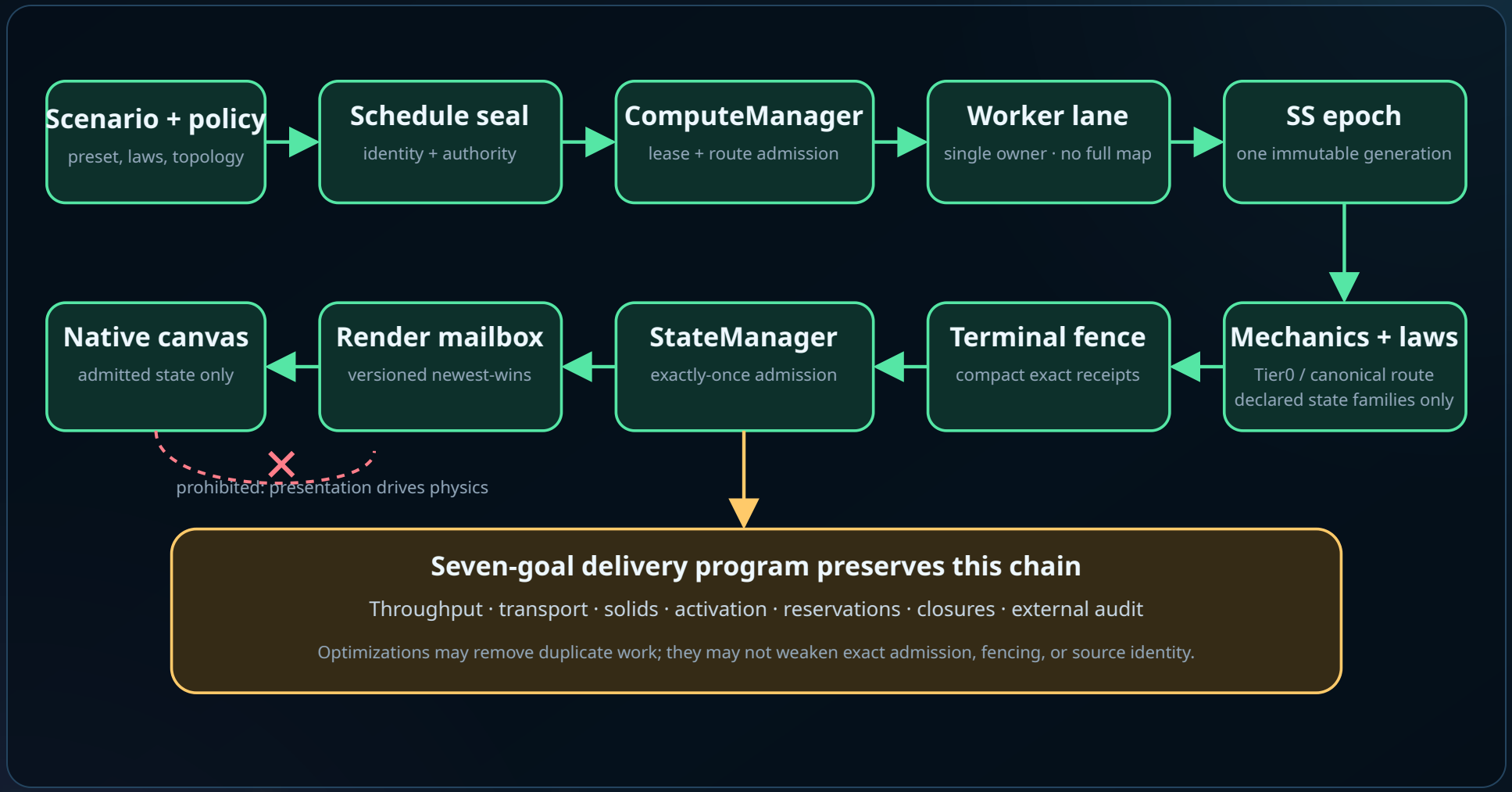
≥ 1×
simulation-time / wall-time target for every preset

7
ordered immediate goals with explicit dependency boundaries

offline
scientific database audit; fuzzy release signal, never runtime authority

The authoritative runtime loop is landed

renderer observes · StateManager admits · worker lane executes



landed and receipt-gated · next implementation boundary · prohibited authority edge

Physics cadence is independent

Rendering consumes admitted outputs; it cannot be required to produce pressure, mechanics, or law state.

One spatial authority

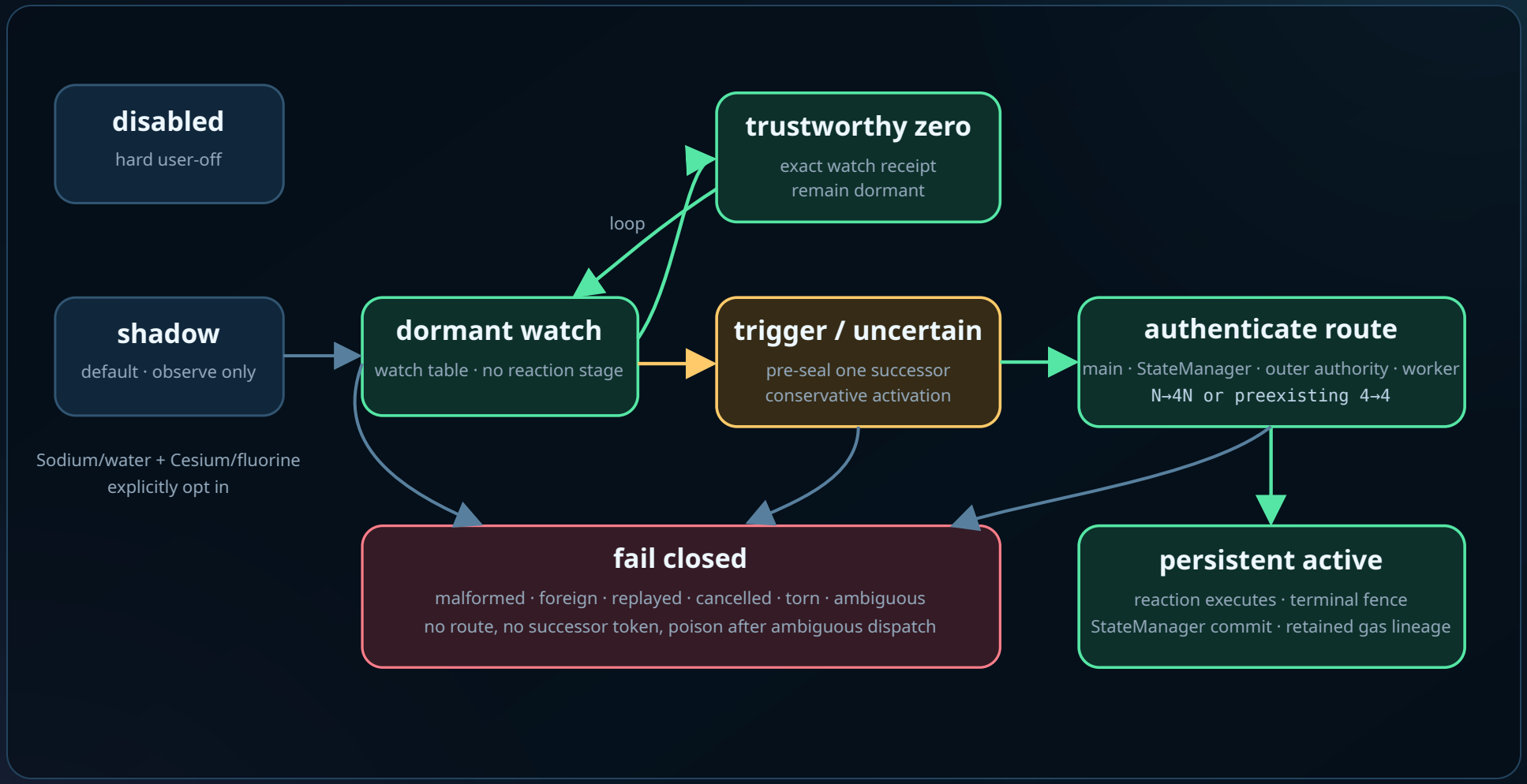
Mechanics, exact-near, cross-level, aggregate, and render views consume the same immutable SS generation.

Fail closed, visibly

Foreign, torn, replayed, ambiguous, unfenced, or uncommitted results never mint successor authority.

Reaction routing is authoritative—within an exact scope

serialized scene/worker lane · reaction family only



IMPLEMENTED

Exact enablement

Activation becomes persistent only after canonical execution, fence completion, exact consumption witnesses, and StateManager commit.

SCOPE BOUNDARY

Not a general router yet

No claim yet for every law family, arbitrary SS nodes, distributed multi-consumer reservation, or online closure derivation.

DELIVERY INVARIANT

Preserve the exact tuple

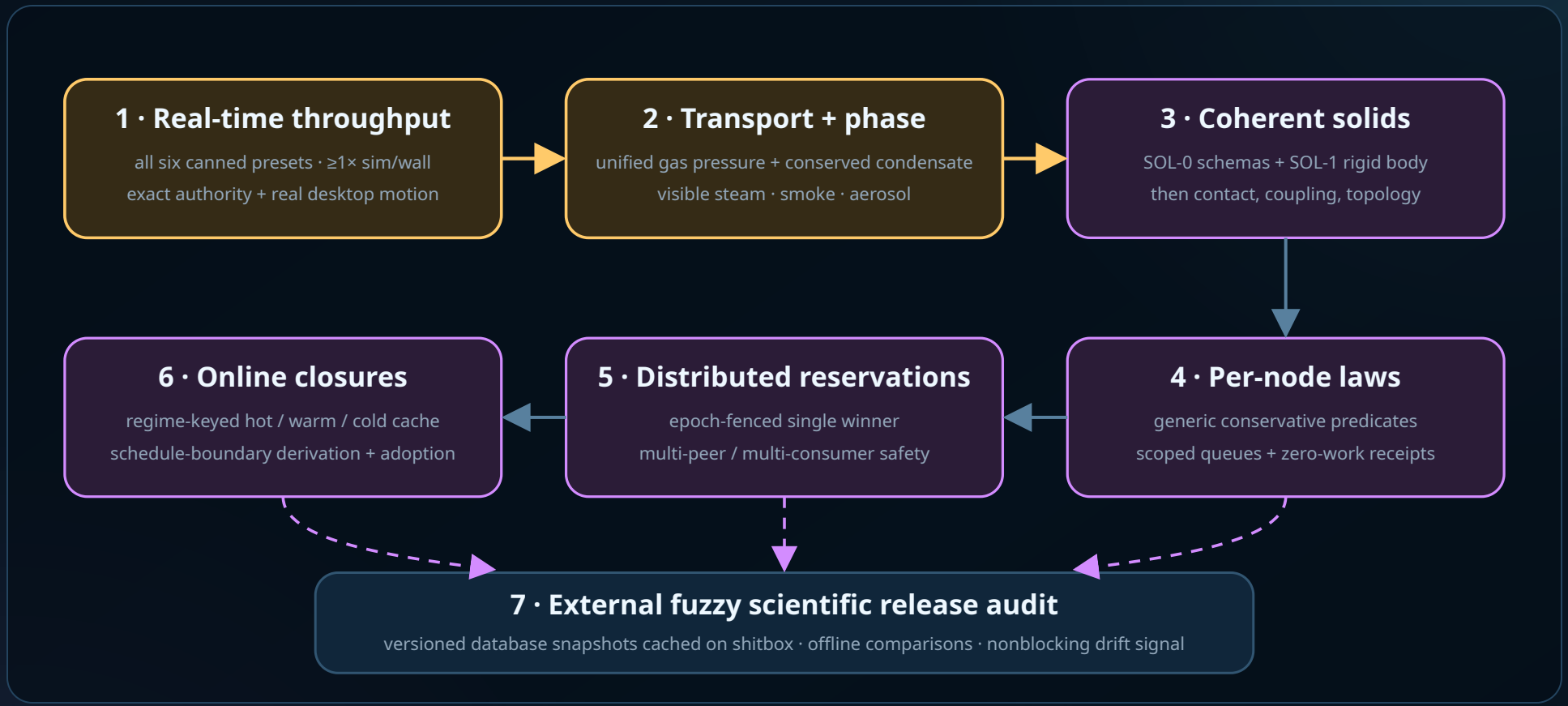
Throughput and later generalization consume the full route and lineage—not raw watch words or visually inferred activation.

Dynamic-law implementation status and boundary

03 · IMMEDIATE PROGRAM

Seven ordered goals restore speed, physical breadth, and scale

runtime work first · external audit last



Current gate

Goal 1 closes only when every preset sustains real time and a real-desktop browser sweep confirms visible animation.

Physical payoff next

Goal 2 makes steam and aerosol visible by conserved phase transport—not presentation-only color or an invented plume force.

Audit is a sidecar

Goal 7 runs around major changes and releases. Scientific drift informs judgment; it never schedules runtime work or blocks admission.

Runtime delivery is serial where dependencies are real

`measure → restore physics → broaden authority`

1 · Throughput	**Substrate** Atomic Tier-0 worker schedules, zero full readback, exact commit/fence, worker-owned native presentation.	**Now** All-six matrix, route telemetry, duplicate-presentation removal, and ≥1× simulation-time / wall-time gates.	**Acceptance** Every canned preset passes measured cadence plus a sequential, visible real-desktop animation inspection.
2 · Transport + phase	**Landed** Sparse mechanics fields, four phase-pure carrier lanes, conservative transfer, monotonic phase-volume rendering.	**Next** Unify live gas occupancy/free-volume pressure, repair product P2G pressure, and conserve condensed aerosol mass.	**Acceptance** Pressure-aware boiling/condensation and physically derived visible steam, mist, and aerosol; H_2 stays optically thin.
3 · Coherent solids	**Prerequisite** Canonical SS exact-near and sparse mechanics substrate. Existing body IDs are not coherent-body authority.	**Then SOL-0 / SOL-1** Body/frame/member schemas, GPU invariants, inertia/wrench state, pose integration, direct resident mesh transform.	**SOL-2…6** Contact, solid-liquid coupling, deformable shape, phase/fracture topology, then multilevel/planetary bodies.
4–6 · Law scale	**Seeds** Per-node queue rows, exact reaction watch, schedule admission, and a regime-keyed closure store.	**Generalize locally** Per-node predicates and scoped invocation; preserve conservative activation and zero-work receipts.	**Then distribute** Epoch-fenced multi-consumer reservation before remote work; wire hot/warm/cold online closure derivation.

HELD

Third SS level

Do not resume until two-level sparsity, conservation, and coherent-solid prerequisites justify it.

GOAL 7 SIDECAR

External fuzzy audit

Compare exported observables with immutable scientific-database snapshots cached on shitbox; report drift without runtime coupling.

OPERATING CADENCE

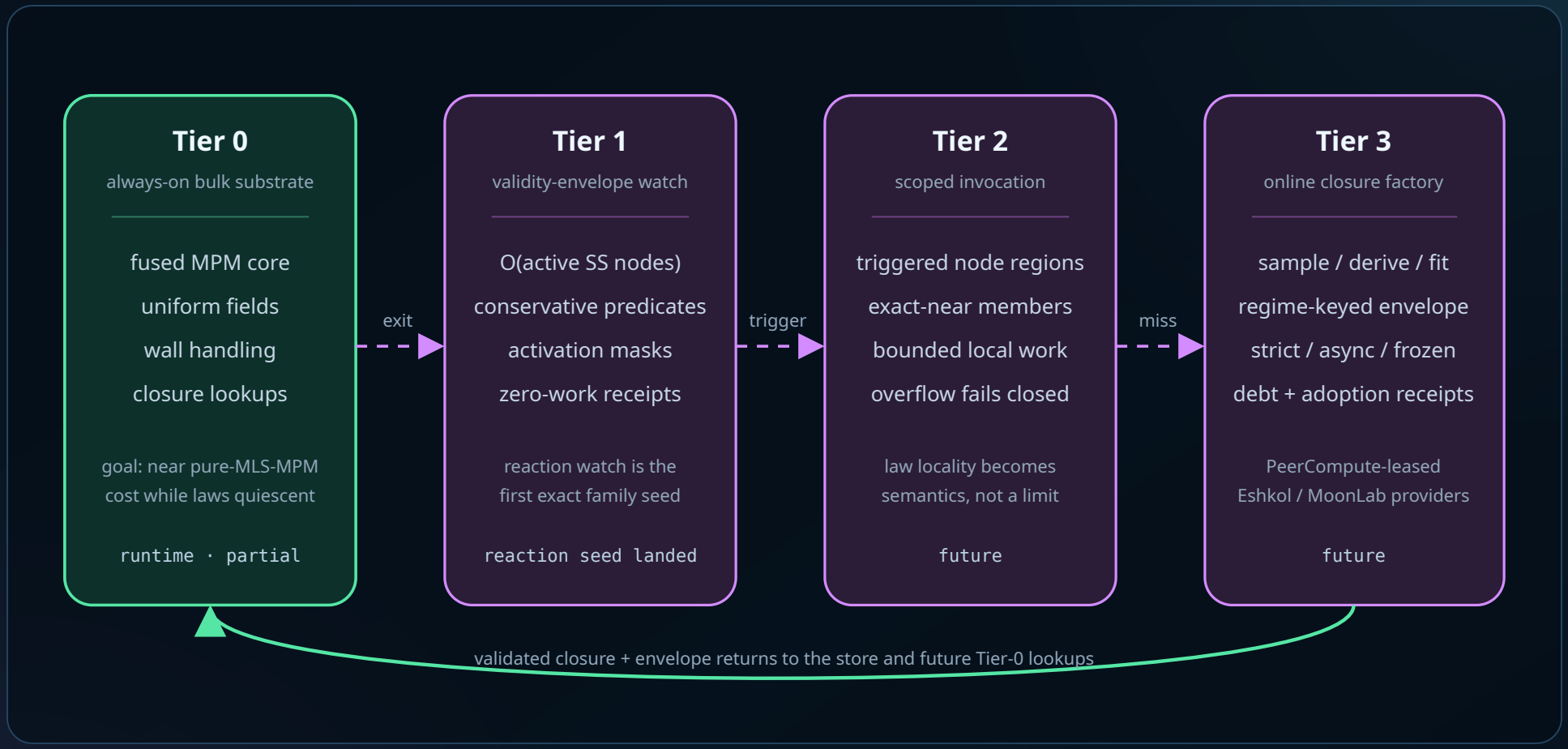
Real desktop evidence

Keep periodic configured-preset checks in visible system Chrome throughout runtime work; close only the owned browser session.

Order: throughput → transport/steam-smoke → solids → per-node laws → distributed reservations → online closures → external fuzzy release audit.

The closure economy scales laws without making them disappear

cache miss = trigger · scale descent = scoped sparsity



Do not remove laws

Inactive laws become cheap through conservative watches, scoped execution, and cached closures—not by deleting their physics.

Do not block the world

Async derivation may continue on declared extrapolation debt, adopting only at schedule boundaries with replayable receipts.

Do not overclaim

A closure carries provenance, validity domain, method/runtime fingerprints, uncertainty, and scientific validation state.

06 · DISPOSITION + GUARDRAILS

Done, moot, held, and still open are different states

honest claims are part of the architecture

DONE

Do not reopen

- canonical SS spatial authority refactor
- worker-owned serialized lane
- reaction activation authority seed
- truthful control-preview labeling and fail-closed startup hold
- native source/presentation correlation

MOOT / SUPERSEDED

Keep as evidence only

- old Slice 9 worktree/server handoff
- July SS routing and delivery queues
- private-bin cubic-barrier follow-ups
- session-operation checkboxes
- June PDF status packet

IMMEDIATE QUEUE

Dependency-ordered, not discarded

- all-preset real-time throughput
- steam/smoke/aerosol transport closure
- coherent solids SOL-0 through SOL-6
- general per-node law routing
- distributed reservations and online closures

Non-negotiable delivery rules

- GPU MLS-MPM / worker-resident route is the primary integration target.
- Normal hot execution stays free of full particle/thermo readback.
- Every runtime acceptance claim names its authority, source, generation, and tolerance.
- Periodic configured presets and random reaction families run in visible desktop Chrome with sequential visual inspection.
- Throughput work cannot weaken physics source correlation, presentation admission, or fail-closed dynamic-law authority.
- Scientific comparison is offline, versioned, fuzzy, and nonblocking; database results never become runtime authority.

Current source-of-truth documents

- `plan/refactor/handoff.md`
- `plan/implementation-status.md`
- `plan/todo/ss-regression.md`
- `plan/todo/scale-adaptive-law-activation-plan.md`
- `plan/todo/surface-contact-transport-phase-closure-plan.md`
- `plan/todo/sol-critic.md`
- `plan/solver-law-inventory.md`
- `plan/todo/README.md`

This packet updates planning status; it does not convert provisional thermochemistry or architecture liveness into calibrated science.

Next action: finish the all-six real-time throughput gate, run the real-desktop visual sweep, then restore physically derived steam and aerosol transport.

Source: plan/ULG_Triad_Current_Plan_Addendum.html